EquaCert: Verifying Completeness Results for Quantum Circuits in Agda

ANONYMOUS AUTHOR(S)

A set of circuit equations is complete if every equation between circuits with the same semantics follows from it, and minimal if no equation can be derived from the others. Proofs of these properties involve long case analyses that can span hundreds of pages, making the details hard to check by hand. To address this, we extend an existing Agda library into EquaCert, a framework that divides the work between humans and machines. Humans set up shared definitions of circuits, matrix semantics, and group presentations, and state each paper's main theorem using them. LLM coding agents then write the proofs following the papers' arguments, and Agda checks them. A human therefore need only compare each formalization's equations, gate semantics, and theorem statement with the paper's. Using EquaCert, we check the main results of sixteen papers, fifteen on completeness and one on minimality. These results cover discrete gate sets such as Clifford, Clifford+T, and Toffoli–Hadamard, on qubits or qudits of every odd prime dimension. We verify fourteen of them and fix some mathematical errors we find in the remaining two. We provide a Lean translation of EquaCert, with its theorems stated over Mathlib's standard definitions. We demonstrate EquaCert's ability to help prove new results by presenting a new complete circuit family: seven equations that are complete for four-bit reversible circuits over {X, CX, CCX}. We expect EquaCert to help with other open completeness problems, especially for quantum circuits in higher dimensions.

1 Introduction

Rewriting of quantum circuits is a standard tool in circuit optimization [41, 53, 54, 68], equivalence checking [69], hardware mapping [66], and reasoning about quantum programs [31]. It relies on an *equational theory* for a family of circuits: a set of equations between circuits, each used as a rewrite rule in either direction. A theory is *sound* if the two sides of every rule have the same semantics, that is, denote the same unitary matrix. Conversely, it is *complete* if any two circuits with the same semantics can be rewritten into each other, so the rules suffice to establish every equivalence. It is *minimal* if no rule can be derived from the others. Studying minimality shows which rules are indispensable, which helps us understand the algebraic structure of a circuit family [16, 23].

Completeness results have been established for a wide range of circuit families, including reversible linear circuits [45, 60], reversible circuits over multiple-control Toffoli gates [32], Clifford [10, 47, 52, 62], CNOT-dihedral [1, 15], two-qubit Clifford+T [12], three-qubit Clifford+CS [13], Toffoli–Hadamard [3], and real Clifford+CH [22] circuits, as well as universal quantum circuits [24, 25]. Completeness has also been obtained for the group of two-level matrices underlying several of these circuit families [11, 21, 31, 36, 48]. Minimality has been shown for six of these families [16] and for universal quantum circuits [23]. Proofs of both properties typically follow a simple strategy, but carrying it out requires long case analyses, with supporting derivations sometimes spanning hundreds of pages [22, 31, 47].

Peer review often does not check proofs of this length in detail. A case can be missed, and a long derivation can use an equation outside the stated rules or contain circular reasoning (i.e., rely, through earlier lemmas, on the equation being derived). A *proof assistant* such as Agda or Lean accepts a proof only if every case is covered and every step follows from the stated rules. Until recently, formalizing these proofs was impractical, given their many lemmas and long derivations. LLM coding agents change this: they can now write proofs that a proof assistant accepts. For example, Anthropic reports that LLMs wrote a 13-million-line Lean proof of Fermat's Last Theorem within 11 days [7]. However, with this great power also comes great responsibility: a proof assistant checks a proof only against the theorem as stated, which may not match the intended result [39].

For example, Bastounis et al. [8] find that the Lean formalization of an announced Navier–Stokes result proves weaker intermediate statements than the written proof, because of mistranslations. So *aligning* the formal statement with the source paper remains a human task. A reader who trusts the proof assistant needs to still check that statement and the definitions it uses, though they do not have to check the full proof.

We present EquaCert (the *Cert*ification of *Equa*tional theories), a framework in Agda [17, 55] for verifying the completeness and minimality of circuit equational theories. In particular, we make the following contributions:

- **A general framework.** EquaCert states every result over one syntax for words and circuits and one definition of completeness, and twelve of its sixteen results over one matrix semantics. It extends the library of Bian and Feng [9] with matrix semantics, three-wire gates, zero-wire gates (scalars such as global phases), PROPs, and independence of relations.
- **Human-checked alignment.** The alignment between the formalized statements in Agda and the text of the paper is checked directly by the authors. This process is made easy by the usage of the standardized framework and the separation of the theorem statement from the proof.
- **Certifications at scale.** We formalize the main results of sixteen papers, fifteen on completeness and one on minimality. Fifteen of the papers are published, and one is an unpublished internship report [21]. These cover a variety of circuit families, from Clifford to Toffoli–Hadamard circuits, and include results using both qubits or higher-dimensional qudits.
- **Corrections.** We find and fix gaps in two published papers: a proof that misses a case and a theorem that omits a relation.
- **New result.** As a demonstration of how EquaCert can be used to find new results, we prove that seven equations are complete for four-bit reversible circuits over {X, CX, CCX}.

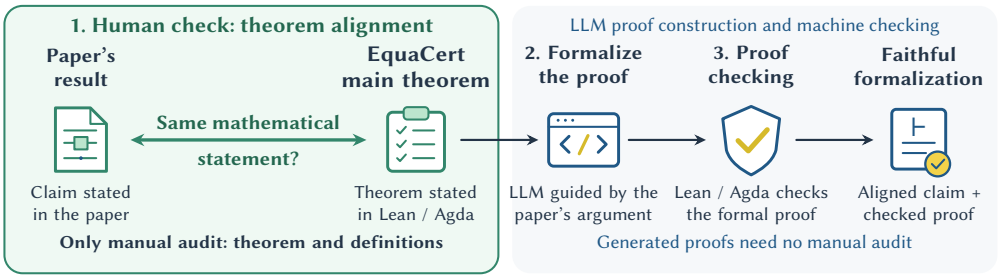


Fig. 1. EquaCert's workflow. Humans check only that the formal theorem and its definitions state the paper's result (step 1); LLM agents write the formal proof, guided by the paper's, and the proof assistant checks it (steps 2 and 3).

Figure 1 shows how EquaCert divides the work. LLM coding agents write the formal statement and its proof, guided by the paper's text, Agda checks the proof, and humans check that the statement says what the paper claims. Because all results share one framework, the dependencies between them are checked as well. We also provide a Lean [29] version of the formalization, for a broader audience.

Related Work. Before the advent of LLMs, work in quantum computing on formalisation has focused on verified compilers and program logics that prove that quantum programs are correct, or that circuit optimizations preserve their meaning [19, 41, 57, 71]. Mechanized libraries for circuit

rewriting, such as VyZX for the ZX-calculus [46], prove the soundness of rewrite rules rather than the completeness of a rule set. Only a few completeness results have been formalized, each in isolation, mostly by its authors. Choudhury et al. [20] formalize in Agda that the equations of the reversible programming language Π are complete for permutations of finite sets. Also in Agda, Bian and Selinger formalize their completeness proofs for two-qubit Clifford+T and three-qubit Clifford+CS circuits [12, 13]. That these results are formalized independently, instead of in a common framework can be problematic because completeness results can build on one another. Bian and Selinger's formalizations assume the underlying matrix-group presentations [11, 36]. Clément's result for real Clifford+CH circuits [22] relies on Hadamard-Π's presentation of $O_n(\mathbb{Z}[1/\sqrt{2}])$ [31], whose proof we show misses a case (Section 5). The completeness of $\sqrt{\Pi}$, a quantum extension of Π [18], rests on the presentations [11, 12, 62], whose relations it derives. A single framework such as EquaCert makes such dependencies checkable, lets reviewers check shared definitions once, and lets formalizations reuse earlier ones.

Recent AI-assisted work has also formalized quantum program logics, algorithms, and complexity results [49, 59, 70]. Several concurrent projects divide the work as we do: humans check the statements and LLM agents write the proofs. MerLean formalizes three quantum computing papers [58], EconCSLib formalizes twenty papers in economics and computation [33], and an LLM agent has proved part of a verified compiler correct [56]. QECert, a Lean library for quantum error correction, likewise gathers many results in one place [40]. The closest to EquaCert is the work of Shirobokov et al. [64] on quantum programs, in which LLM agents write both the programs and their proofs.

Structure of the paper. Section 2 reviews equational theories and the main techniques for proving their completeness and minimality. Section 3 introduces EquaCert. Sections 4 and 5 present the formalized results and the corrections to the source papers. Section 6 gives the new result, a complete equational theory for four-bit reversible circuits over {X, CX, CCX}, and explains how its proof is constructed and checked; Appendix E gives the full proof. Section 7 discusses limitations and future work. The remaining appendices give background, an introduction to the framework on permutation circuits, implementation details, the comparison with Lean, and the details of the corrections.

2 Preliminaries

EquaCert verifies the completeness and minimality of rule sets over gates or multi-level matrices. Section 2.1 introduces the main gate sets, their matrix groups, and the multi-level matrices that generate those groups. Section 2.2 reviews how completeness and minimality are proved.

2.1 Circuit Families and Their Matrix Groups

Most of the circuit families we formalize are restricted Clifford+T circuits. Amy et al. [2] show that several of these circuit families represent exactly the unitary matrices over a subring of $\mathbb{Z}\left[1/\sqrt{2}, i\right]$, with at most one ancilla. Appendix A defines the restricted gate sets, the underlying rings and matrix groups, and what it means for a circuit to represent a matrix with ancillas. These finitely-generated rings are helpful for us, because in Agda we can then encode each element of $\mathbb{Z}\left[1/\sqrt{2}, i\right]$ and of its subrings $\mathbb{Z}[1/2, i]$, $\mathbb{Z}\left[1/\sqrt{2}\right]$, and $\mathbb{Z}[1/2]$ exactly, as a few integers with no rounding. For example, $(a + b\sqrt{2})/\sqrt{2}^k \in \mathbb{Z}\left[1/\sqrt{2}\right]$ is encoded as the integers a, b, and k.[1]

[1] Circuits with continuous parameters, such as the universal circuits of [25], have matrix entries that cannot be encoded this way. See Section 7, where we discuss the limitations of EquaCert and future work.

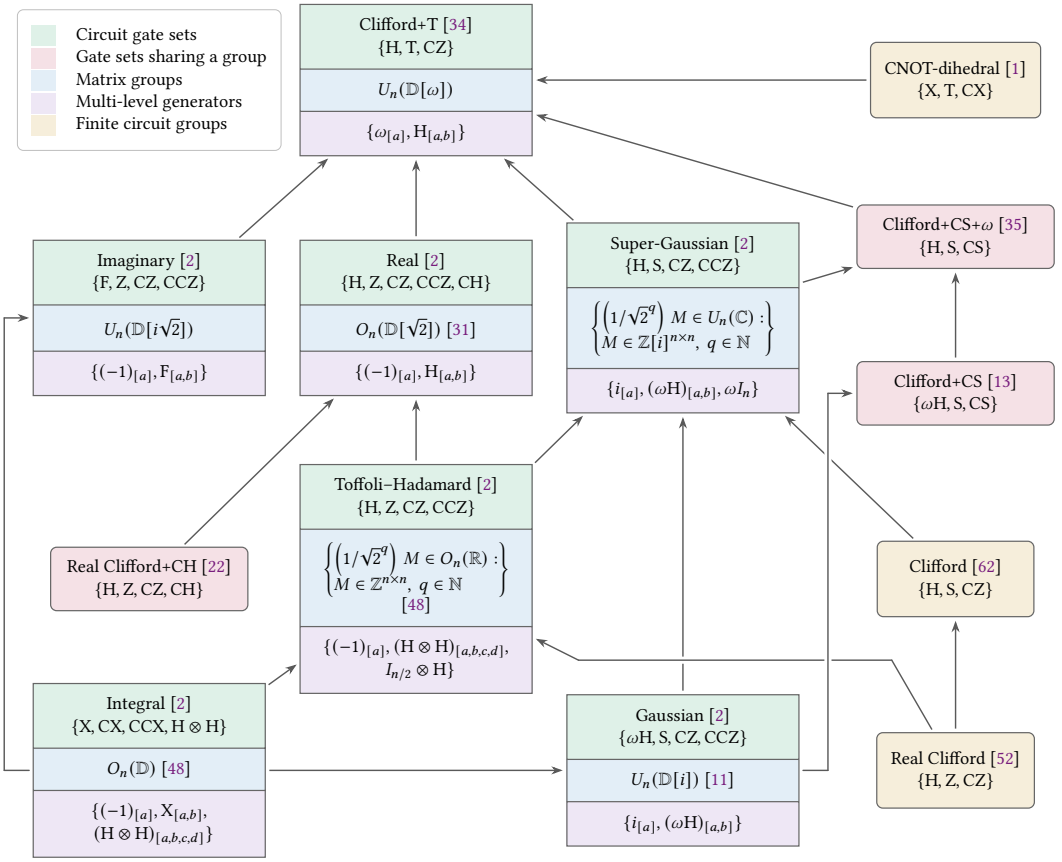


Fig. 2. Restricted Clifford+T circuit families and their underlying matrix groups, where k is the number of qubits and $n = 2^k$ is the dimension. The circuits over a gate set represent exactly the matrices of the group below it when allowing for at most one clean ancilla. Arrows denote inclusion. Here $\mathbb{D} = \mathbb{Z}[1/2]$, $\omega = e^{i\pi/4}$, and $\mathrm{F} = (\mathrm{H} + iI)/\sqrt{2}$. Clifford+CS+$\omega$ denotes adjoining the scalar ω to Clifford+CS.

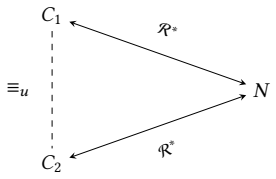
Figure 2 displays the restricted Clifford+T gate sets, their matrix groups, and multi-level generators (defined below). An arrow from one family to another means that every matrix represented by an ancilla-free circuit of the first is also represented by one of the second. If both boxes show a matrix group, then the first group is a subgroup of the second. Section A defines the remaining notation and gives the source of each matrix group.

For an $m \times m$ matrix M and distinct indices $a_1, \ldots, a_m$ in $\{0, \ldots, n-1\}$, the *m-level* matrix $M_{[a_1,\ldots,a_m]}$ is the $n \times n$ matrix that acts as M on the standard basis vectors $e_{a_1}, \ldots, e_{a_m}$ and as the identity on the others. For example, $(-1)_{[a]}$ negates e_a, and $\mathrm{X}_{[a,b]}$ exchanges e_a and e_b. Two generators in Figure 2, $I_{n/2} \otimes \mathrm{H}$ and ωI_n, are not multi-level matrices, since they act on every basis vector. Completeness is studied at both levels of the figure, for rule sets over its gate sets and for rule sets over its multi-level generators.

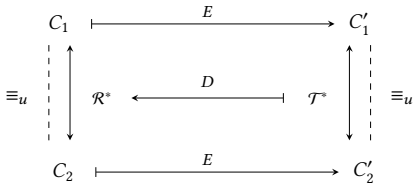
2.2 Proof Strategies for Completeness and Minimality

A circuit is a word over an alphabet $\mathcal{G}$ of gates. A rewrite rule is then a relation between words over this alphabet, and a rule set $\mathcal{R}$ generates a congruence on circuits.[2] We associate an *interpretation* $[\![C]\!]$ to each circuit C, namely its underlying matrix. A sound and complete rule set is then a *presentation* of the group of matrices that circuits denote, with the gates as generators and the rules as relations.

There are two main strategies for proving completeness of a rule set $\mathcal{R}$ over a gate set $\mathcal{G}$, both supported in EquaCert. Consider any two circuits C_1 and C_2 over $\mathcal{G}$ that have the same interpretation, $[\![C_1]\!] = [\![C_2]\!]$. We need to show that C_1 can be rewritten into C_2 using $\mathcal{R}$. Figure 3a outlines the first strategy. This strategy specifies a unique normal form for each matrix [1, 10, 15, 32, 45, 47, 52, 60, 62], often by giving an exact synthesis algorithm [11, 21, 31, 36, 48], which writes each matrix as a word over the generators. Completeness follows by proving that every circuit can be transformed into its normal form using the rewrite rules. By soundness and uniqueness, C_1 and C_2 have the same normal form N. Reversing the path from C_2 to N and concatenating it with the path from C_1 to N then gives the desired path of rewrites.



(a) The rewrites from C_1 and C_2 to their common normal form N using $\mathcal{R}$ give rewrites from C_1 to C_2 using $\mathcal{R}$.



(b) Completeness of $\mathcal{T}$ gives rewrites between $C_1' = E(C_1)$ and $C_2' = E(C_2)$. Translating this proof back gives rewrites between C_1 and C_2 using $\mathcal{R}$.

Fig. 3. Two strategies for proving completeness of $\mathcal{R}$ over a gate set $\mathcal{G}$. $\overset{\mathcal{R}^*}{\longleftrightarrow}$ means applying a finite sequence of rewrites using $\mathcal{R}$ in either direction, where $*$ denotes zero or more steps. Dashed $\equiv_u$ lines assert equal matrices.

The exact synthesis algorithms used by the first strategy consider groups of $n \times n$ matrices, such as $O_n(\mathbb{Z}[1/2])$ and $U_n(\mathbb{Z}[1/\sqrt{2}, i])$, rather than circuits (the first block of Table 1). Their presentations by m-level matrices are stated and proved for every n. Circuits on k qubits only reach the dimensions $n = 2^k$, so a circuit completeness result built on such a presentation uses only the case $n = 2^k$. For completeness proofs based on exact synthesis, the induction argument is the same in every formalization: one shared module in the Lean version, and a copy of about 200 lines in each Agda formalization. So each such proof only has to relate the steps of its own algorithm, which reduces a matrix to the identity, to the rewrite rules. This separates the algorithm-specific case analysis, the main work in each formalization, from the common completeness proof. Section 3.1 expands on the inductive proof and states precisely what each formalization must prove.

Figure 3b illustrates the second strategy, which uses an already complete rule set $\mathcal{T}$ over a different gate set $\mathcal{G}'$ [3, 12–14, 16, 22, 24, 25, 30, 61]. Here, E and D translate circuits from $\mathcal{G}$ to $\mathcal{G}'$ and back, replacing each gate with a circuit over the other gate set. Both E and D preserve circuit composition and the underlying matrix. Hence $C_1' = E(C_1)$ and $C_2' = E(C_2)$ have the same

[2] A congruence on circuits is an equivalence relation (i.e., reflexive, symmetric, and transitive) that is also compatible with composition. The congruence generated by $\mathcal{R}$, written $\overset{\mathcal{R}^*}{\longleftrightarrow}$, is the smallest such relation that contains every rule of $\mathcal{R}$.

matrix, so completeness of $\mathcal{T}$ gives $C_1' \overset{\mathcal{T}^*}{\longleftrightarrow} C_2'$. To transfer completeness from $\mathcal{T}$ to $\mathcal{R}$, we first show that every rule of $\mathcal{T}$, translated by D, can be derived using $\mathcal{R}$. We can then replace each step of these rewrites by its translated derivation, giving $D(C_1') \overset{\mathcal{R}^*}{\longleftrightarrow} D(C_2')$. It then remains to derive $C_j \overset{\mathcal{R}^*}{\longleftrightarrow} D(C_j')$ for $j \in \{1, 2\}$, for which it suffices to derive the round-trip rewrites $g \overset{\mathcal{R}^*}{\longleftrightarrow} D(E(g))$ for each $g \in \mathcal{G}$. Sometimes the two strategies are combined, as in Li et al. [48], who extend their presentation of $O_n(\mathbb{Z}[1/2])$ to the group L_n of orthogonal matrices $N/\sqrt{2}^j$, where N is an integer matrix and $j \geq 0$.

Starting from a complete rule set, we can remove a redundant rule by deriving it from the other rules, thereby preserving completeness. To show that the resulting rule set is minimal it suffices to give for each remaining rule r another compositional interpretation of circuits (not necessarily into matrices) that every other rule preserves but r does not. Any derivation using the other rules preserves this interpretation, so r cannot be derived from them [16, 23].

3 The EquaCert Framework

Our Agda framework EquaCert states every completeness theorem as a group presentation. Appendix B introduces it on its smallest complete example, the swap circuits that present the symmetric groups. Gates are indexed by their arity: the number of wires on which they act. Each source relation is stated on its original wires, and shared *structural rules* allow it to be used inside a larger circuit. These structural rules are independent of the gate set and make the circuit structure explicit that source papers often leave implicit.

We associate to circuits a matrix semantics $[\![-]\!]$ with entries in a finitely generated ring such as $\mathbb{Z}\left[1/\sqrt{2}, i\right]$. This defines a group homomorphism to a unitary matrix group G, whose group structure is checked once per ring using Agda's standard library. The interpretation extends from gates to circuits, and the structural rules are sound for every gate set. The Lean version and the Agda semantics of the six minimality fragments prove this once; the other Agda formalizations prove the structural cases for their own gates from lemmas on Kronecker products. Beyond that, each formalization checks only its own rules.

Soundness and completeness say the relations present the image of $[\![-]\!]$, which is the subgroup of G generated by the gates. In Agda, mathematical structures such as groups are defined as records. The record $\mathcal{R}$ IsSubPresentationOf G contains the semantics $[\![-]\!]$, with proofs of soundness and completeness. The record $\mathcal{R}$ IsPresentationOf G adds surjectivity, so the relations present all of G.

Figure 4 summarizes the two obligations: checking each rule's matrix equality for soundness, and deriving a known complete rule set $\mathcal{T}$ from $\mathcal{R}$ for completeness.

Each formalization ends with one of these records. For example, Statement (1) states that the real stabilizer relations [52] present the subgroup generated by their gates, while Statement (2) states that the Hadamard-Π relations [31] present the specified orthogonal group. The fields in Statements (3) and (4) state the two directions: equal matrices imply a derivable circuit equation, and every derivable equation preserves the matrix.

```
presentation : ∀ n → (n CRel,_===_) IsSubPresentationOf U (2 ^ n)          (1)
presentation : ∀ n → (n MRel,_===_) IsPresentationOf (O n)                 (2)
complete : ∀ {n} {w v : Circuit n} → [[ w ]] ≈ᵁ [[ v ]] → ⊢ w ≈ v          (3)
sound : ∀ {n} {w v : Circuit n} → ⊢ w ≈ v → [[ w ]] ≈ᵁ [[ v ]]             (4)
```

The minimality statement in Statement (5) says that the rule set is complete on n wires and becomes incomplete there if any one of its rules stated on exactly n wires is removed [16]. For three

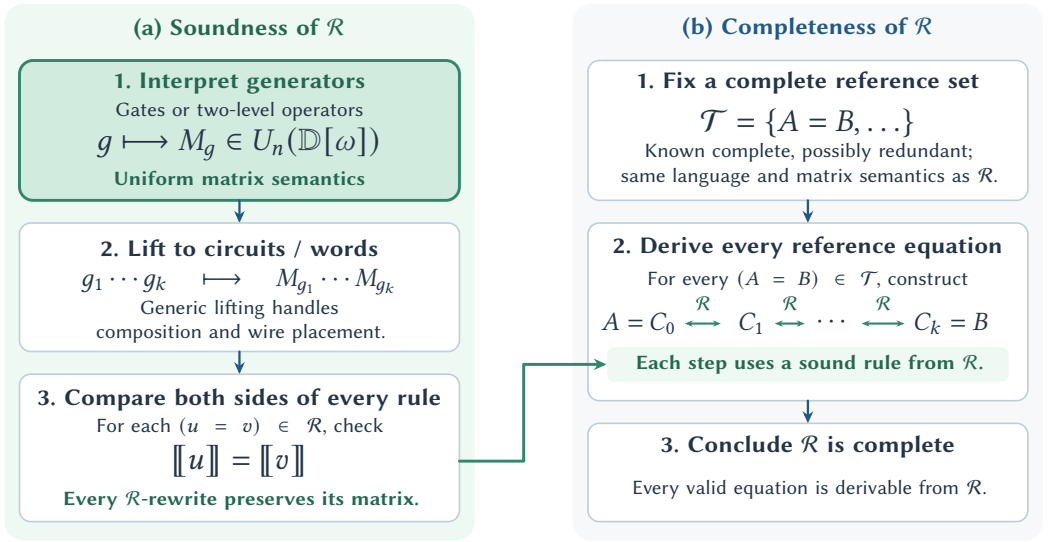


Fig. 4. EquaCert's shared matrix interpretation supports both proof obligations (soundness and completeness). The left column checks every rule of $\mathcal{R}$; the right derives every rule of a known complete set $\mathcal{T}$ in the same circuit language and matrix semantics. Each rewrite step uses a rule of $\mathcal{R}$, whose soundness is checked on the left. The displayed $U_n(\mathbb{D}[\omega])$ covers qubit fragments [2], with $\mathbb{D} = \mathbb{Z}[1/2]$ and $\omega = e^{i\pi/4}$. Qudit circuit families use other rings.

of the six fragments, the source leaves open completeness beyond a certain number of wires or the independence of some rules, and the formal statements also specify bounds on n (Section 4).

```
minimal-complete : ∀ {n} → (n ⊢_===_) IsMinimal × (n ⊢_===_) IsComplete
```
(5)

For each presentation, a reader checks four components against the source paper: the transcribed relations, the target group, the matrix of each gate, and the assignment of $[\![-]\!]$ to the record. Each Agda formalization collects these components and its main theorem in a `MainTheorem` module. The file `MainTheorems.agda` restates the results; the corresponding Lean statements are in each formalization's `Theorem` module, which `QCert/Table1.lean` imports (`QCert` is the folder of the Lean library).

Wire Placement and Structural Rules. Wires are numbered from 0 at the bottom, and a gate is defined as acting on the bottom wires of a circuit. We define a shift operator which places a circuit on n wires onto wires $1, \ldots, n$ of a circuit with $n+1$ wires. In the matrix semantics, this corresponds to taking the tensor product with the identity. This shift operation is defined once for all gate sets.

Each source relation is stated on as many wires as the paper draws it on. Structural rules allow an equation to be shifted by one wire, gates on disjoint wires to commute, and a scalar to be placed on any wire. EquaCert extends the earlier framework's one- and two-wire syntax with three-wire gates and their structural rule. Scalars are zero-wire generators; their placement rule allows exact equations to account for global phase.

In the earlier framework, swap is an ordinary gate whose laws must be included in each rule set. EquaCert also supports a PROP presentation, in which a designated two-wire gate is the swap and the framework supplies its involution law and the laws for moving gates across it. The source rule set then contains only its own relations, each on its original wires; swap and its laws are supplied

by the framework. We use this representation for the circuits of [1, 15, 16], and for the reversible circuits of Section 6, whose structural rules are drawn in Figure 10. Appendix C defines PROPs and explains the theorem notation.

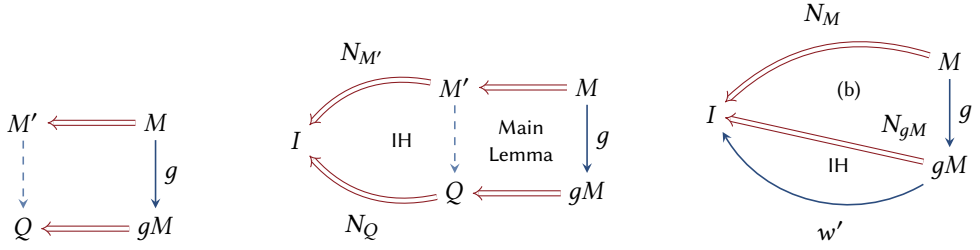
Relation to the Earlier Library. The library of Bian and Feng [9] has 74 Agda modules and about 15,200 lines of code. Its framework, 59 modules and about 8,600 lines, has words, presentations, normal forms, the Reidemeister–Schreier method, and presentations of direct, semidirect, and amalgamated products and of group extensions, with the symmetric, cyclic, and Pauli groups as examples. Its circuits have one- and two-wire gates and three structural rules. It has no matrix semantics, and it contains none of the results of Table 1. EquaCert's framework has 92 modules and about 13,200 lines: 45 modules come from the earlier library, 14 of them unchanged, and 47 are new. The additions are zero-wire and three-wire gates with their structural rules, PROPs with the laws of the swap, independence of relations, the transport of presentations along embeddings and translations, and the tactic library of [12, 13]. The matrix semantics and the unitary and orthogonal groups that the formalizations share are new as well; they are defined in the formalizations of Table 1 and are counted there.

3.1 Reusable Completeness Arguments

For the matrix groups discussed in Section 2, normal forms come from exact synthesis, and the proof that every circuit can be transformed into its normal form has the same structure for each algorithm. Each matrix, or *state*, has a *level*, built from quantities such as the least denominator exponent of a column, and the algorithm reduces each state M to the identity in steps that lower the level. These steps form the normal word N_M.

The *Main Lemma* says that for every state M and generator g, a diagram of relations closes the square formed by g and the algorithm's next steps from M and gM, passing only through states of lower level (Figure 5a).

By well-founded induction on the level, the relations then rewrite $N_{gM}\, g$ into N_M (Figure 5b), and by induction on length, every word that reduces M to the identity rewrites into N_M (Figure 5c). Two words with the same matrix therefore rewrite into each other. These inductions have the same form for every presentation, so the main work in each formalization is its Main Lemma.



(a) The Main Lemma. (b) Induction on the level. (c) Induction on length.

Fig. 5. Completeness from the Main Lemma. Plain arrows are words, double arrows are steps or runs of the algorithm, and dashed arrows are words through states of lower level.

3.2 Formalization Workflow

A formalization is carried out by a coding agent in sessions that a human directs, in six steps. They refine the three steps of Figure 1, whose human check is the sixth step here. *(1) Fix the target.* We

name the paper and the result to be proved, in one sentence to the agent. The framework fixes the form of the statement: a set of relations, a group, and a semantics. *(2) Check the source.* The agent transcribes the paper's rewrite rules and gate matrices. It checks the rules, the intermediate results, and the lines of the derivations with exact matrices in short scripts, before and while it writes Agda code. These checks, and proof steps that failed in Agda, found the misprints of Appendix F. *(3) Prove.* The agent writes the proof, guided by the paper's. For a long derivation, it writes a script that turns each line of the paper into one step of the Agda proof, so a failing step points to a specific line of the paper; where the paper gives no derivation, a script searches for one. *(4) Check.* Agda checks every module. A step that is not yet proved is a parameter of a module, never a postulate, so each theorem shows what it still assumes. *(5) Record.* An agent carries little from one session to the next, so what it learns is written to a notes file in the repository that every later session reads: the conventions of each formalization, the cost of checking each module, and the mistakes not to repeat. *(6) Read the statements.* At the end, each formalization gets a `MainTheorem` module that states its theorem with no unproved hypothesis, in one form for all formalizations. We read these statements against the papers: the relations, the target group, and the gate matrices. By our count, the statements and the definitions they mention take about 8,200 of the 377,000 lines of Agda code, about 2% (Appendix C.2). Where a formalization has gate matrices, Agda also decides every rule against them by computation.

The history of the repository gives the scale of this work. The fifteen formalizations other than qudit Clifford were written and first proved between 19 September and 16 October 2026, in about 400 commits. Judging by the times of their commits, ten of the fifteen reached their main theorem less than a day after their first commit. Leaving out a port of existing Agda code and two whose first commit is already large, seven did, in between one and a half and ten hours. The largest, real Clifford+CH, took more than three weeks. Making the proofs fast to check and giving them statements of one form took until 8 November.

Matrix Equality Checking. The main computational cost came from Agda's conversion checker. It accepts syntactically identical terms immediately; otherwise, it evaluates them to compare their results. For long matrix products, this evaluation can take exponential time. We keep circuit interpretations abstract and decide matrix equality by computing a single Boolean using row operations. For these products, this is orders of magnitude faster than comparing matrices entry by entry. The notes file records this technique, so later formalizations avoid the same slowdowns.

Where the Workflow Failed. The agents' mistakes resembled human ones: a step taken from a garbled copy of a page, two products taken in the wrong order, a hypothesis stated more strongly than its source states it. A computation or a failed proof caught each of these, not the agent's own reading, and not always at once. The two products agree on three qubits, where an Agda decision procedure had checked the rules that use them, and differ from four qubits onward; a simulation found the error within six days, while a later result was being proved. The hypothesis was a theorem that the paper imports from another paper. It had been stated for all words over the generators, including degenerate generators that the source excludes, and in that form it is false, so every theorem that assumed it was vacuous. The false hypothesis stood for about eighteen days, until it was to be proved. Both errors were in intermediate statements, and neither was found by reading. A final theorem with no unproved hypothesis is safe from both kinds: a false hypothesis cannot remain in it, and a wrong auxiliary definition stops its proof. What Agda cannot check is the final statement itself against the paper. That is the sixth step, and it is done by a human.

The work also cost more than its final size shows. Three formalizations first took between one and three and a half hours to check, and were reworked. The first proof of the theorem of Section 6

was replaced by another within a day. Usage limits ended sessions in the middle of their work more than once.

AI Use. The library of [9] is the starting point of the framework, and the proofs of [12, 13] are ported from their authors' Agda code; agents revised both. The rest was written by LLM coding agents: the extensions of the framework, the Agda proofs, and the scripts that search for and generate derivations, in the workflow above, and the Lean version, from the source papers. The exception is the qudit Clifford formalization, which predates this workflow; agents wrote parts of it. Most of the agents' work was done by Claude Code, with several versions of Anthropic's Claude models. A second coding agent, Astra, worked on the Toffoli–Hadamard formalization of Table 1 and on parts of the Lean version. Claude Code also helped to draft and revise the text of this paper, which we checked.

3.3 Avoiding Completeness Lost in Translation

For a formalization to be useful, its formal statement must say what the source paper claims. We make this connection explicit in three ways.

First, we define the target group independently of the circuit syntax, using matrices or an algebraic construction. A completeness theorem then relates the circuit equations to a semantics that does not depend on them, rather than to a group built from the equations themselves.

Second, when a source rule set must be changed, the formal theorem names the revised set. This lets a reader tell a repair to the proof apart from a correction to the theorem itself (Section 5).

Finally, general completeness arguments and group constructions are verified once, among them the Reidemeister–Schreier method [51] and presentations of semidirect products, both inherited from [9]. Each formalization that uses them only needs to prove their hypotheses and check the connection to its own circuit semantics.

4 Formalized Completeness and Minimality Proofs

The sixteen formalizations in Table 1 contain about 377,000 lines of Agda code: about 204,000 written directly, 107,000 generated by scripts (Section 3.2), and 66,000 ported from the Agda proofs of [12, 13]. Every module is checked with Agda's `--safe` flag, so no proof uses a postulate. Except as qualified below, each theorem has the scope of the result in its source paper, that is, the same gate set and the same widths or dimensions. Moreover, all matrices are computed exactly. Their entries lie in algebraic number rings, whose elements are given by finitely many integers, so Agda checks automatically whether two matrices are equal. Over the real or complex numbers, no such automatic check exists, which limits the equality checking of EquaCert to discrete gate sets, as we discuss in Section 7.

Figure 6 adds completeness status to the families and inclusions of Figure 2. It separates circuit relations from presentations of the underlying matrix groups. EquaCert covers the integral, scaled dyadic, Gaussian, and real matrix presentations, but circuit completeness for Clifford+T and Clifford+CS is still restricted to small widths. A presentation of the ambient matrix group does not by itself provide relations for every circuit subgroup.

The green checks in Figure 6 correspond to the qubit part of Table 1; the qudit results and minimality proofs are also listed in the table. The asterisks connect this coverage to the source corrections in Section 5.

Four qualifications apply to the Agda results in Table 1. The two-qubit Clifford+T and three-qubit Clifford+CS formalizations [12, 13] reuse the authors' Agda proofs and connect them to EquaCert's shared matrix presentations (Section 3). The qudit Clifford theorem of Bian, Li et al. [10] presents an independently constructed algebraic model, not a group of matrices; that this model is the

Source	Object	Result	Proof	Agda Mod	Agda time (s)	Lean time (s)
Groups of m-level matrices						
Greylyn [2014]	$U_n(\mathbb{Z}[1/\sqrt{2}, i])$	presentation, $n \leq 4$	S	34	37.28	145.34
Bian and Selinger [2021]	$U_n(\mathbb{Z}[1/2, i])$	presentation	S	39	53.73	126.99
Li et al. [2021]	$O_n(\mathbb{Z}[1/2])$, L_n	presentations	S	71	84.52	185.32
Fang et al. [2026]	$O_n(\mathbb{Z}[1/\sqrt{2}])$	presentation	S	46	48.33	231.57
Clément [2019]	$O_n(\mathbb{Z}[1/\sqrt{2}])$	presentation	S	43	67.72	51.99
Circuits						
Selinger [2015]	Clifford	sub-presentation	S	32	35.01	88.24
Bian, Li et al. [2026]	Clifford, odd prime p	presentation of a model	E	216	499.37	197.32
Makary et al. [2021]	real stabilizer	sub-presentation	S	31	37.94	79.75
Amy et al. [2018]	CNOT-dihedral	sub-presentation	S	31	21.24	101.22
Blake [2026b]	phase-affine, prime d	sub-presentations	S	57	83.30	257.24
Bian and Selinger [2023a]	two-qubit Clifford+T	sub-presentation in U_4	P	63	135.79	356.79
Bian and Selinger [2023b]	three-qubit Clifford+CS	sub-presentation in U_8	P	125	298.29	1330.89
Clément [2026]	real Clifford+CH	sub-presentation	S	348	419.99	5478.79
Amy et al. [2024]	three-qubit Toffoli–Hadamard	presentation of L_8	R	203	111.17	134.91
Li et al. [2025]	qutrit Clifford	presentation	S	90	285.58	179.60
Minimality						
Blake [2026c]	six fragments	minimal, complete[†]	S	80	180.10	358.87
Total				1,509	2,399.36	9,304.83

Table 1. Completeness and minimality results formalized in EquaCert. A presentation gives generators and relations for a whole group, and a sub-presentation for the subgroup generated by the gate set named in the Object column of the Circuits block. L_n is the group of orthogonal matrices $N/\sqrt{2}^j$, where N is an integer matrix and $j \geq 0$. Appendix A gives precise definitions of the rings, matrix groups, and gates. Mod counts Agda modules. The Agda times are typechecking times from scratch with Agda 2.8, the times its profiler reports for the row's modules; the Lean times are elapsed folder-build times. Both were measured on the same machine, under the conditions in Appendix D. Both columns exclude the shared library and other formalizations on which a row depends. The Result column describes the Agda statements; Appendix D explains the semantic differences. The Proof column gives the origin of the Agda proof: S, written by coding agents, guided by the proof in the source; P, ported from the Agda proof of the source's authors; R, a replay of the derivation files that accompany the source; E, begun before the workflow of Section 3.2, with parts written by agents. Appendix C.2 gives each row's size and what it uses. [†]For three of the fragments, up to the numbers of wires given in the text. The source of the fifth row is an unpublished internship report.

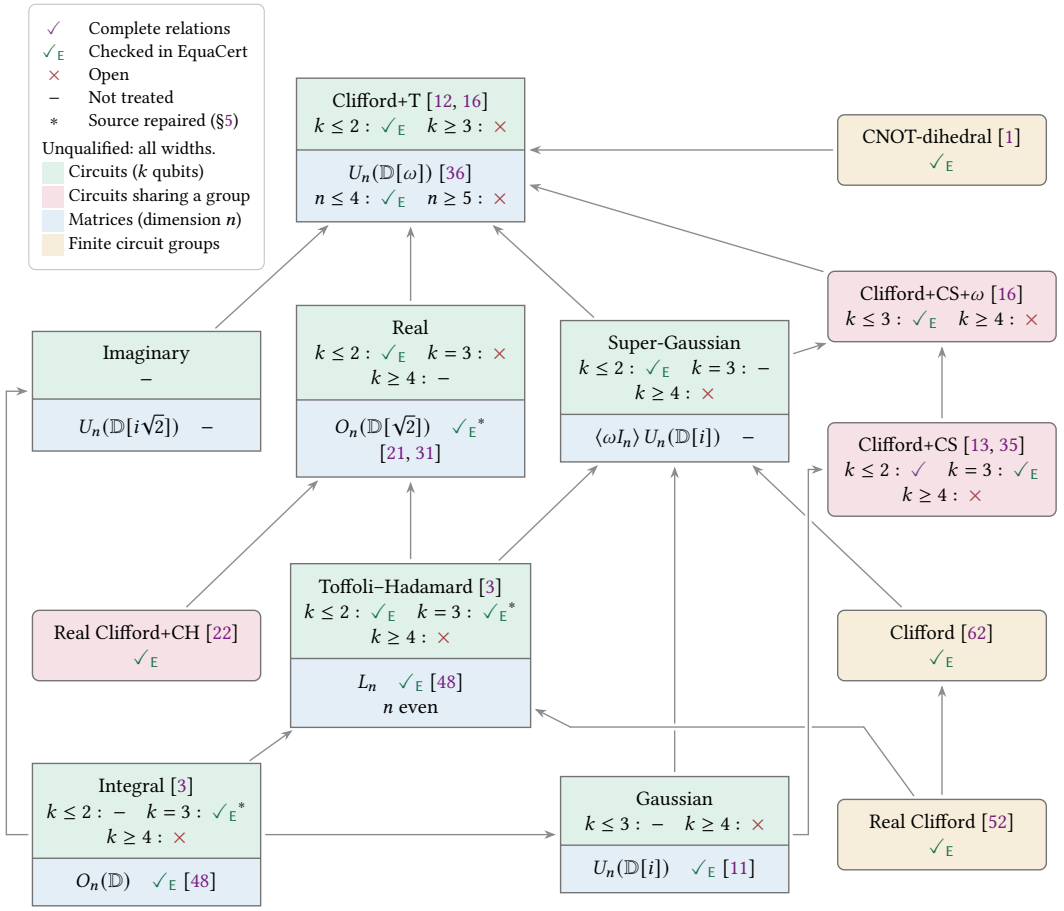


Fig. 6. Completeness progress on the same families and inclusion arrows as Figure 2. Gates and multi-level generators are omitted; upper compartments concern ancilla-free circuits, lower compartments matrix presentations. Here k counts qubits and n is matrix dimension ($n = 2^k$ for a circuit). A plain check does not assert EquaCert verification, and a dash marks a case that EquaCert does not treat, including consequences of a checked result for another gate set. The higher-dimensional Clifford+T matrix presentation remains incomplete. L_n is defined in Table 1; the other notation is as in Figure 2 and Appendix A.

group of unitary Clifford operators is classical mathematics [37, 38] that the Agda formalization does not check. The Toffoli–Hadamard row measures a formalization that replays, as generated certificates, the derivations in the files that accompany the source. A second formalization, whose derivations are found by search, proves the same presentation, that of $O_8(\mathbb{Z}[1/2])$ on which it rests, and the independence of the relation restored in Section 5 [6]. In the minimality row, three of the six fragments are minimal and complete on every number of wires. The other three carry bounds: qutrit Clifford is complete on every number of wires, with every rule on at most two wires shown necessary; Clifford+T is complete on at most two wires, with every rule on at most one shown necessary; and Clifford+CS is complete on at most three, with every rule on at most two shown necessary.

4.1 Comparison of the Agda and Lean Formalizations

The Lean [29] version of EquaCert formalizes the same sixteen results on top of Mathlib [67]. Its framework is a translation of the Agda framework, and, judging by its commits, coding agents then wrote its sixteen formalizations from the source papers within a week. It came second, and we present it as a second version of the same results and not as a second contribution of the same kind. Its point is its theorem statements. They rest on Mathlib's standard definitions, with matrix groups over Mathlib's real or complex numbers in place of our exact rings, so a reader can check them against the source papers using definitions that are already widely used and reviewed. For qudit Clifford, the Lean statement also removes an argument that the Agda statement leaves to external mathematics (Section 4). The Lean proofs use no `sorry` and no `native_decide`, and the main theorems depend only on Lean's three standard axioms (Appendix D).

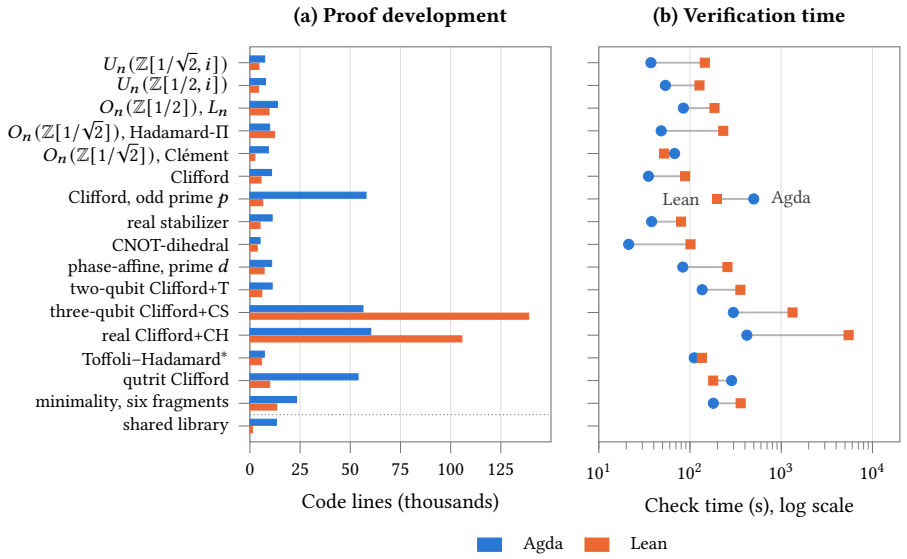


Fig. 7. The Agda and Lean formalizations of each row of Table 1. (a) Code lines, excluding blank lines and comments. For Toffoli–Hadamard* it shows another pair, one proof in both assistants (Appendix D): a second Agda proof and its translation to Lean. The last row is the shared library of each framework. (b) The check times of Table 1, Agda (circles) and Lean (squares), measured on the same machine, with methods described in Appendix D.

Since the two versions share the design of their framework, their agreement does not independently validate it. Figure 7 describes the two versions. The two proof assistants can be compared only where both check the same proof. Two formalizations are translated module for module: a second proof of the Toffoli–Hadamard result, which is the row marked $*$ in panel (a), and the four-bit theorem of Section 6 (Appendix D). There the two versions need about the same amount of text, and Lean takes 3.4 and 10 times as long as Agda to check them, in the second case mostly to load Mathlib once for each of its 54 modules. In the other rows the proofs differ, and so do the two ways of measuring time, so apart from that row panels (a) and (b) describe the two formalizations and not the assistants: the Lean formalization is shorter in thirteen rows, and the Agda one is checked faster in thirteen. Agda's normal forms are executable functions, whereas Lean's are existence results or `noncomputable` definitions. Both versions can run a procedure that decides

whether two Clifford circuits have the same matrix. They implement it differently, and Lean's is faster from three qubits onward (Figure 14 in Appendix D). What a reader should take from the comparison is that the two versions state the same sixteen theorems over different definitions, that the same proof needs about the same amount of text in both assistants, and that Lean takes several times as long to check it; the differences between the rows of Table 1 come from the proofs. Appendix D gives the methods and the limits of each measurement, summarized in Table 4.

5 Corrections to the Source Papers

The formalizations in Table 1 identify two kinds of issues in the source papers. Most are typos, and some of them appear in statements that the proofs rely on. These are easy to fix, and Appendix F lists them by source. Two others are harder to repair. One is a gap in a proof, and the other is a relation missing from a theorem. We describe both below, and Appendix F gives the counterexamples to the lemmas and the interpretation that separates the missing relation.

The gap is in the proof of Theorem 4.14 of Hadamard-Π [31], which states that the relations in its Figure 6, over the one- and two-level matrices $(-1)_{[a]}$, $\mathrm{X}_{[a,b]}$, and $\mathrm{H}_{[a,b]}$, present the group $O_n\left(\mathbb{Z}\left[1/\sqrt{2}\right]\right)$. One case of the proof relies on Lemmas A.16 and A.17, which have counterexamples in dimension 6, and the case analysis built on them misses a matrix in dimension 9. Their main result is however still correct: Clément's unpublished internship report [21] presents the same group with different relations, and his case analysis covers the missing case, using which the formalization of Theorem 4.14 is completed. We also prove that the two sets of relations are equivalent in every dimension and formalize Clément's proof [5].

The relation is missing from Theorem 5.2 of [3]. It states that the relations in its Figures 4 and 7, over the Toffoli and $\mathrm{K} = \mathrm{H} \otimes \mathrm{H}$ gates, present the group $O_8\left(\mathbb{Z}\left[1/2\right]\right)$. This is the first of two steps toward presenting the scaled dyadic orthogonal group L_8 by three-qubit Toffoli–Hadamard circuits. The relation r_{K} in its Section 5.5, which splits K on two qubits into two copies controlled by the remaining qubit, one for each of its values, is in neither figure. Nor can it be derived from them. Hence, Theorem 5.2 fails as stated. Our formalization [6] shows that r_{K} is independent by giving an interpretation of circuits that every other relation preserves but r_{K} does not, and proves the presentations of $O_8\left(\mathbb{Z}\left[1/2\right]\right)$ and L_8 with r_{K} restored.

6 A New Result: Complete Equations for Four-Bit Reversible Circuits

Beyond the results of Table 1, we use EquaCert to prove and check a new completeness theorem for classical reversible circuits, that is, quantum circuits whose gates permute the computational basis. Without ancillas, circuits over the gates X (NOT), CX (CNOT), and CCX (Toffoli) realize every permutation of the 2^k bit strings on $k \leq 3$ wires, and exactly the even ones on $k \geq 4$ wires [63]. Complete equational theories are known for linear reversible circuits [45, 60] and for circuits over multiple-control Toffoli gates [32]. Complete sets of identities are also known for circuits with ancillary bits, over CNOT [27] and over the Toffoli gate [26], where circuits denote partial maps, and over multiple-control Toffoli gates [43]. In these theories, a derivation between two ancilla-free circuits may pass through circuits with ancillas, so they do not directly give a complete theory for the ancilla-free circuits, which form a group on each number of wires. We give one for $\{\mathrm{X}, \mathrm{CX}, \mathrm{CCX}\}$ on at most four wires: the theory $\mathcal{T}$ of the seven equations in Figure 9. We do not know whether the seven equations are independent. Whether they remain complete on five or more wires is ongoing work (Section 7).

6.1 Rules and Completeness

Notation. Wires are numbered from 0 at the bottom, and a *state* of k wires is a string of k bits, that of the top wire first. A circuit is a product of the gates X, CX, and CCX and of swaps of adjacent wires, and a circuit u on k wires denotes a permutation $[\![u]\!]$ of the 2^k states. We write products of circuits and of permutations in diagrammatic order, meaning that pq applies p first and then q. Two circuits are *equal modulo* $\mathcal{T}$ if the equations of Figure 9 and the structural rules of Figure 10 rewrite one into the other. In circuit diagrams, time runs from left to right, $\oplus$ is a NOT target, crossed wires are the structural swap, and a slashed wire is a bundle of wires (Figure 8a). A filled control tests 1, and an open control tests 0 and stands for a filled control between two X gates on its wire (Figure 8b).



(a) A slashed wire is a bundle of wires.

(b) An open control is a filled control between two X gates on its wire.

Fig. 8. Two conventions of the circuit diagrams in Section 6.

A CNOT or Toffoli acts on adjacent wires with its target below its controls, and one drawn elsewhere is that gate conjugated by swaps. In Lafont's diagrams (Figure 11), each wire is instead a point and each grey box is a cycle. Following Lafont, ω_i is the cycle of the points i, $i+1$, and $i+2$, a stairs $s_d(j)$ is a product of consecutive cycles that carries d to j, and a canonical form is a product of stairs $s_d(j_d)$, one for each d, whose targets are the j_d. On four wires, the points are the sixteen states, which we number along the reflected Gray code and call positions. The circuit Ω_i carries out ω_i on the positions, and $\mathrm{Can}(j_0, \ldots, j_{13})$ is the canonical form with these targets, read as a circuit. In the Agda statements, $\vdash^{\mathrm{R}}$ denotes equality modulo $\mathcal{T}$, $\doteq$ equality of the functions $[\![w]\!]$ at every state, `canon x` the canonical circuit with the targets `x`, and `nf w` the tuple of targets of the permutation that `w` carries out (`nf-sem` of the module `NormalForm`).

Theorem 6.1 (soundness and completeness). *On any number of wires, two circuits that are equal modulo* $\mathcal{T}$ *denote the same permutation. Conversely, for* $1 \le k \le 4$*, two circuits on* k *wires that denote the same permutation are equal modulo* $\mathcal{T}$*, and the circuits on* k *wires modulo* $\mathcal{T}$ *form a group that* $u \mapsto [\![u]\!]$ *maps isomorphically onto the symmetric group* $\mathrm{Sym}(2^k)$ *for* $k \le 3$ *and onto the alternating group* $\mathrm{Alt}(16)$ *for* $k = 4$.

6.2 Canonical Circuits

On four wires, every circuit also has a normal form: Lafont's canonical form of an even permutation [45], read as a circuit. Lafont generates the even permutations of N *points*, which we number $0, \ldots, N-1$, by the *cycles* $\omega_0, \ldots, \omega_{N-3}$, where ω_i carries i to $i+1$, $i+1$ to $i+2$, and $i+2$ to i (Figure 11(a)). The *stairs* from d to j, for $d \le N-3$, is the word $s_d(j) = \omega_d\, \omega_{d+1} \cdots \omega_{j-1}$ for $d \le j \le N-2$, and $s_d(N-1) = s_d(N-2)\, \omega_{N-3}$; it carries d to j and fixes the points below d. A *canonical form* is a product of stairs $s_{N-3}(j_{N-3}) \cdots s_1(j_1)\, s_0(j_0)$ with $d \le j_d \le N-1$ (Figure 11(b)). Every even permutation π is the permutation of exactly one canonical form [45, Lemma 6]; its *targets* $j_0, \ldots, j_{N-3}$ are found in turn: j_d is the image of d under $\pi\, s_0(j_0)^{-1} \cdots s_{d-1}(j_{d-1})^{-1}$.

Here the points are the sixteen states of four wires, numbered along the reflected Gray code: *position* i is the state with the binary digits of $i \oplus \lfloor i/2 \rfloor$, and a circuit *carries out* a permutation of the

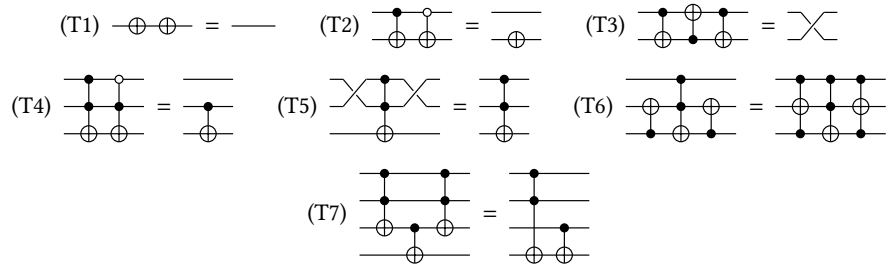


Fig. 9. The theory $\mathcal{T}$ for circuits over $\{\mathrm{X}, \mathrm{CX}, \mathrm{CCX}\}$. Time runs left to right, and wire 0 is the bottom wire. A filled control tests 1, an open control tests 0 (it is the filled one conjugated by X on its wire), $\oplus$ is a NOT target, and crossed wires are the structural swap. A CNOT or Toffoli drawn in another position, as in (T3), (T6), and (T7), is the gate on adjacent wires with its target below its controls, conjugated by swaps. Each equation can be used in either direction and, by the structural rules of Figure 10, on any wires of a circuit, adjacent or not and in any order.

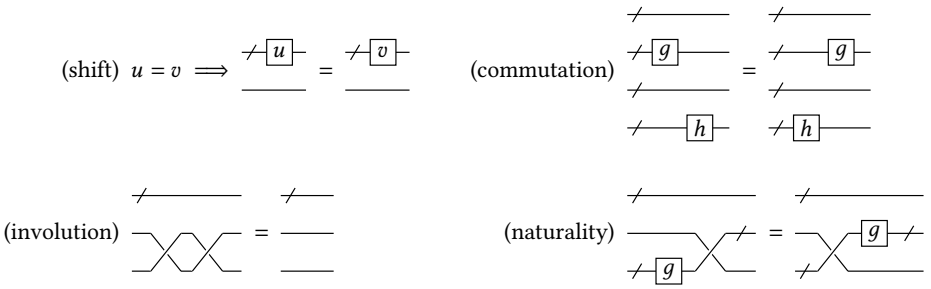


Fig. 10. The structural rules for circuits over $\{\mathrm{X}, \mathrm{CX}, \mathrm{CCX}\}$, which EquaCert supplies for every gate set with a structural swap (Section 3). Here u and v are circuits, g and h are X, CX, CCX, or the swap, and a slashed wire carries the wires of the box on it or, if it has none, any number of idle wires. An equation also holds with an idle wire below it, a gate commutes with any gate above it, the swap is an involution, and a gate slides past a wire that crosses all of its wires; for g the swap, (naturality) is the braid relation. (shift) adds wires only below, while the other rules hold under any idle wires.

positions. Consecutive positions differ on one wire, so the cycle ω_i is carried out by a circuit Ω_i that is equal modulo $\mathcal{T}$ to a commutator of two Toffoli gates with filled or open controls (Figure 12(a), Appendices E.3 and E.8). Writing Ω_i for ω_i turns the canonical form with the targets $j_0, \ldots, j_{13}$ into the *canonical circuit* $\mathrm{Can}(j_0, \ldots, j_{13})$.

Theorem 6.2 (normal form). *Every four-wire circuit u is equal modulo $\mathcal{T}$ to exactly one canonical circuit, the one with the targets of the permutation that u carries out. Moreover, $(j_0, \ldots, j_{13}) \mapsto [\![\mathrm{Can}(j_0, \ldots, j_{13})]\!]$ is a bijection from the $16!/2$ tuples of targets onto $\mathrm{Alt}(16)$.*

For example, the Toffoli gate of Figure 12(b) exchanges the positions 4 and 5 and the positions 10 and 11. The canonical form of this permutation is $\omega_9\,\omega_8\,\omega_7\,\omega_6\,\omega_5\,\omega_4$, six stairs of one cycle each (Figure 11(c)), so the gate is equal modulo $\mathcal{T}$ to the canonical circuit $\Omega_9\,\Omega_8\,\Omega_7\,\Omega_6\,\Omega_5\,\Omega_4$ (Appendix E.11).

Both theorems are proved in Agda, and Statements (6) to (10) are from the MainTheorem module of the formalization [4].

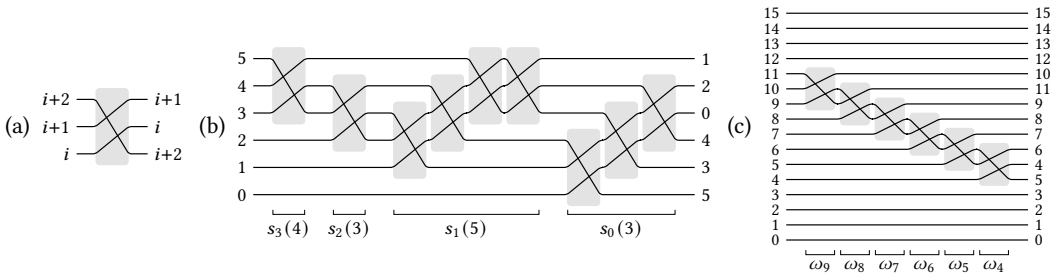


Fig. 11. Lafont's canonical forms as circuits whose wires are the points, after his drawings. Time runs left to right, the point 0 is the bottom wire, a grey box is a cycle, and each strand is labelled with its point at both ends, so it ends at the height of the image of its point. (a) The cycle ω_i. (b) The canonical form $s_3(4)\,s_2(3)\,s_1(5)\,s_0(3)$ on six points. The stairs $s_d(j)$ raises the strand of the point d to the height j, one wire at each cycle, and the stairs to the last point, $s_1(5)$, repeats its last cycle. (c) The canonical form $\omega_9\,\omega_8\,\omega_7\,\omega_6\,\omega_5\,\omega_4$ on the sixteen positions, which exchanges 4 with 5 and 10 with 11, as the Toffoli gate of Figure 12(b) does.

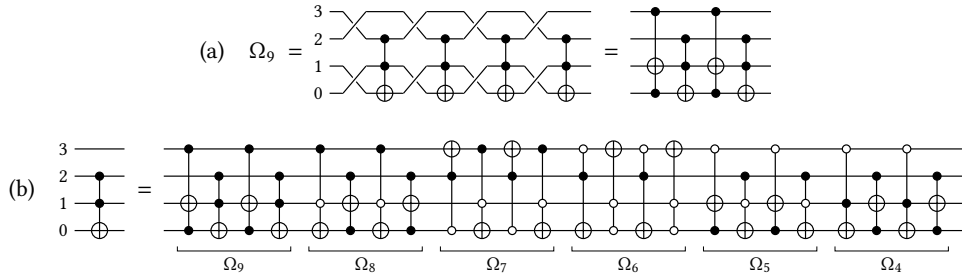


Fig. 12. The circuit Ω_9 and a canonical circuit, drawn as in Figure 9, with the wires numbered at the left. (a) By definition, Ω_9 is two swaps and then a Toffoli gate, repeated four times. Modulo $\mathcal{T}$ it is equal to a commutator of two Toffoli gates. (b) A Toffoli gate and the canonical circuit $\Omega_9\,\Omega_8\,\Omega_7\,\Omega_6\,\Omega_5\,\Omega_4$ that it is equal to modulo $\mathcal{T}$, the translation of Figure 11(c). Each Ω_i is drawn as a commutator of two Toffoli gates, with filled or open controls, that is equal to it modulo $\mathcal{T}$ (Appendix E.8).

```
soundᴿ : ∀ {n} {w v : Circuit n} → ⊢ᴿ w ≈ v → [[ w ]] ≐ [[ v ]]                (6)
completeᴿ : ∀ {w v : Circuit 4} → [[ w ]] ≐ [[ v ]] → ⊢ᴿ w ≈ v                 (7)
presentationᴿ : (4 RRel,_===_) IsPresentationOf Alt-group 16                    (8)
normal-formᴿ : (w : Circuit 4) → ⊢ᴿ canon (nf w) ≈ w                            (9)
canon-unique : ∀ {x y : NF} → [[ canon x ]] ≐ [[ canon y ]] → x ≡ y             (10)
```

6.3 Constructing and Checking the Proof

The proof (Appendix E) follows the second strategy of Figure 3. Here $\mathcal{T}$ is the rule set to be proved complete, and Lafont's presentation is the known complete one. By two changes of presentation, each verified by the framework's theorem for passing between two presentations of one group, the single-coset case of its Reidemeister–Schreier theorem [51] (Section 3.3, Appendix E.1), it derives, from $\mathcal{T}$, Lafont's complete presentation of Alt(16) by the cycles [45]; the library of EquaCert proves that presentation and his lemmas on canonical forms. Read on circuits, his presentation becomes 111 equations: his 105 relations between the Ω_i, and six equations that write each gate and each swap as a product of the Ω_i.

Proof Organization. The proof has five parts.

(1) Lafont's presentation of Alt(16) is carried to the four-wire circuits by a first step. Its relations on the circuits form the *translated relation*, which is therefore complete (Appendices E.2–E.4).
(2) On at most three wires, $\mathcal{T}$ is complete, by a step from the Coxeter presentation of the symmetric group (Appendix E.5).
(3) A four-wire derivation is built from circuits on three adjacent wires, which the second part decides, and from thirteen instances of the rules on all four wires. Two lemmas about circuits of X gates and swaps make it possible to move a gate to any wires (Appendix E.6).
(4) The equations of the translated relation are derived from $\mathcal{T}$: the relations between the cycles, by derivations that a search finds, and then the six equations for the gates (Appendices E.7–E.9).
(5) A second step identifies the two congruences on four wires, which gives Theorem 6.1, and Lafont's canonical forms give Theorem 6.2 (Appendices E.10 and E.11).

No derivation is written without a script except for the short ones behind the two lemmas of the third part. The first and the last part rest on semantics and on Lafont's theorem, and the derivations of the second and the fourth are found by search.

Checked Derivations. Derivations are checked by a small proof engine. An equation is a pair of lists of generators with a derivation between their circuits. A new equation is derived by a list of steps, each of which rewrites an occurrence of one side of a known equation, at a given place in the list, into the other side, and Agda checks the derivation by running the steps. The derivations are generated by scripts that search for them and that replay every step before they write it. A fact is *decided by evaluation* when it is stated as a Boolean that Agda's type checker computes to be true.

Agda replays the 15,607 steps of the four-wire derivations in 20 seconds.

Implementation. The formalization consists of 38 modules in `Examples/Groups/Alt16-RevCir`, all reached from MainTheorem, and every module is checked with Agda's `--safe` flag. Given the interfaces of the shared library, Agda checks them in about 43 seconds of wall time, of which 28 seconds are spent in the modules themselves, the quantity that Table 1 reports for its rows. Of its 28,000 lines, blank lines and comments included, 4,300 are written without a script, and the rest are generated derivations: 2,900 lines on two and three wires, and 21,000 lines on four, in 788 equations with 15,607 steps. In a generated module a list of generators is a string with one character for each generator, and the Boolean that decides a step is a hidden argument that Agda fills in once it has evaluated to true.

The scripts that find the derivations are part of the formalization. They are the Knuth–Bendix completion on two and three wires, the search for the pair lemmas of Proposition E.9, which takes 20 seconds, the best-first search for the relators of Proposition E.11 and Lemma E.12, which takes at most a minute for each, and a compiler from macro derivations to steps of the engine, which replays every step. Each search is bounded, and runs within 1.5 GB. Two changes brought the last four relators within this bound: another form of one cycle for three of them, and a compact storage of the words that the search has seen for the fourth.

The formalization is translated to Lean module for module, with the same generated derivations: its 38 modules, and sixteen more for what it imports from the library of EquaCert, Lafont's presentation among it (Appendix D).

7 Discussion and Future Work

We formalize the results of sixteen papers in EquaCert using one syntax for words and circuits, one definition of group presentations, and shared libraries for rings and their matrix groups. Unifying

these definitions has two benefits. First, presentations of the same group from different papers can be compared and proved equivalent. Second, a new completeness proof can reuse much of the existing work. When completeness follows from an exact synthesis algorithm, the induction that turns the algorithm into a proof carries over with little change, so the main new work is the case analysis over matrices and generators. That case analysis is the most error-prone part of the proof, so it is where machine checking matters most. Reusing these libraries, we also prove a new result, a complete equational theory for four-bit reversible circuits over {X, CX, CCX}.

We are now using EquaCert to extend two completeness results to any number of wires. The first is the four-bit theory of Section 6. For $n \geq 4$, circuits over {X, CX, CCX} realize exactly the even permutations [63], and related complete rule sets use cycles instead of gates [45] or allow multiple-control Toffoli gates [32]. The route of Section 6.3 does not depend on the number of wires: Lafont's presentation covers every number of points, and the first and the last part of the proof are semantic. What a further wire needs is a derivation, from the rules, of Lafont's relations between the cycles, of which there are 465 on five wires where there are 105 on four. We do not know whether the seven equations suffice for them. The second is the three-qubit Toffoli–Hadamard theory [3], which we extend using the presentation of $O_n(\mathbb{Z}[1/2])$ [48]. More broadly, completeness is still open for many other discrete gate sets, especially in higher dimensions, and we plan to study Clifford circuits in all dimensions with EquaCert. Beyond completeness, the exact synthesis libraries can be reused for T-count optimization of multi-qubit Clifford+T circuits.

At the moment, not every completeness result fits into EquaCert, as its scope is limited in two ways. First, it supports only discrete gate sets, so it does not cover completeness results for circuits with continuous parameters, such as rotations by arbitrary angles [23–25]. Second, control is not a circuit constructor in EquaCert, so it does not cover results that treat control as a primitive operation on circuits [14, 30, 61]. We plan to extend EquaCert past both limitations.

Data-Availability Statement

The Agda framework and the sixteen formalizations, the proof of Section 6, the Lean version, the scripts that search for and generate derivations, and the measurement scripts and records behind Table 1 and Figure 7 are submitted with this paper as anonymized supplementary material. The files `MainTheorems.agda` and `QCert/Table1.lean` restate the theorems of Table 1. The proofs are checked with Agda 2.8.0 and version 2.4 of its standard library, and with Lean 4.30.0 and a pinned revision of Mathlib; on the machine of Appendix D, the Agda check takes about an hour and the Lean build under three. We intend to submit the artifact for Artifact Evaluation and to release it under an open-source license.

References

[1] Matthew Amy, Jianxin Chen, and Neil J. Ross. 2018. A finite presentation of CNOT-dihedral operators. In *Proceedings of the 14th International Conference on Quantum Physics and Logic (QPL 2017) (Electronic Proceedings in Theoretical Computer Science, Vol. 266)*. 84–97. arXiv:1701.00140 doi:10.4204/EPTCS.266.5

[2] Matthew Amy, Andrew N. Glaudell, and Neil J. Ross. 2020. Number-theoretic characterizations of some restricted Clifford+T circuits. *Quantum* 4 (2020), 252. doi:10.22331/q-2020-04-06-252

[3] Matthew Amy, Neil J. Ross, and Scott Wesley. 2024. A sound and complete equational theory for 3-qubit Toffoli–Hadamard circuits. In *Proceedings of the 21st International Conference on Quantum Physics and Logic (QPL 2024) (Electronic Proceedings in Theoretical Computer Science, Vol. 406)*. 1–43. arXiv:2407.11152 doi:10.4204/EPTCS.406.1

[4] Anonymous. 2026. EquaCert: Complete equations for four-bit reversible circuits from Lafont's presentation of the alternating group, formalized in Agda and Lean. Supplementary material, module `Alt16-RevCir/MainTheorem` (completeness on one to four wires, the presentations of $\mathrm{Sym}(2^k)$ and $\mathrm{Alt}(16)$, the two changes of presentation, and the canonical form), and its Lean translation `QCert/Alt16RevCir/MainTheorem`.

[5] Anonymous. 2026. EquaCert: The presentations of $O_n(\mathbb{Z}[1/\sqrt{2}])$ by Hadamard-Π and by Clément, formalized in Agda. Supplementary material, modules `Real-Clifford+CH-TwoLevel/Equivalence` (the equivalence of the two

sets of relations) and `Real-Clifford+CH-TwoLevel-Clement` (Clément's proof and its decision trees).

[6] Anonymous. 2026. EquaCert: Three-qubit Toffoli–Hadamard circuits with the relation r_{K}, formalized in Agda. Supplementary material, modules `CCX+H-3qubit-RS/Theorems` (the presentations of $O_8(\mathbb{Z}[1/2])$ and L_8) and `CCX+H-3qubit-RS/Independence` (the interpretation showing that r_{K} is not derivable).

[7] Anthropic. 2026. Formalizing Fermat's Last Theorem. Research post, 4 September 2026. https://www.anthropic.com/research/formalizing-fermats-last-theorem

[8] Alexander Bastounis, Fabian Circelli, and Anders C. Hansen. 2026. Navier–Stokes lost in translation: Why Lean verification of AI autoformalisation does not guarantee correct natural language proofs. arXiv:2610.08144

[9] Xiaoning Bian and Yuan Feng. 2027. Normal forms and complete relations for circuits in Agda. *Proceedings of the ACM on Programming Languages* 11, POPL (2027). To appear.

[10] Xiaoning Bian, Sarah Meng Li, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2026. A complete and natural rule set for multi-qudit Clifford circuits in all odd prime dimensions. arXiv:2609.40106

[11] Xiaoning Bian and Peter Selinger. 2021. Generators and relations for $U_n(\mathbb{Z}[1/2, i])$. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 145–164. arXiv:2105.14047 doi:10.4204/EPTCS.343.8

[12] Xiaoning Bian and Peter Selinger. 2023. Generators and relations for 2-qubit Clifford+T operators. In *Proceedings of the 19th International Conference on Quantum Physics and Logic (QPL 2022) (Electronic Proceedings in Theoretical Computer Science, Vol. 394)*. 13–28. arXiv:2204.02217 doi:10.4204/EPTCS.394.2

[13] Xiaoning Bian and Peter Selinger. 2023. Generators and relations for 3-qubit Clifford+CS operators. In *Proceedings of the 20th International Conference on Quantum Physics and Logic (QPL 2023) (Electronic Proceedings in Theoretical Computer Science, Vol. 384)*. 114–126. arXiv:2306.08530 doi:10.4204/EPTCS.384.7

[14] Colin Blake. 2026. A complete equational presentation of qudit circuits via polycontrolled PROPs. In *Proceedings of the 51st International Symposium on Mathematical Foundations of Computer Science (MFCS 2026) (Leibniz International Proceedings in Informatics, Vol. 386)*. 6:1–6:18. arXiv:2602.09873 doi:10.4230/LIPIcs.MFCS.2026.6

[15] Colin Blake. 2026. Completeness for prime-dimensional phase-affine circuits. In *Proceedings of the 23rd International Conference on Quantum Physics and Logic (QPL 2026) (Electronic Proceedings in Theoretical Computer Science, Vol. 456)*. 40–127. doi:10.4204/EPTCS.456.3

[16] Colin Blake. 2026. Simpler presentations for many fragments of quantum circuits. In *Proceedings of the 11th International Conference on Formal Structures for Computation and Deduction (FSCD 2026) (Leibniz International Proceedings in Informatics, Vol. 378)*. 6:1–6:20. arXiv:2602.09874 doi:10.4230/LIPIcs.FSCD.2026.6

[17] Ana Bove, Peter Dybjer, and Ulf Norell. 2009. A brief overview of Agda – a functional language with dependent types. In *Proceedings of the 22nd International Conference on Theorem Proving in Higher Order Logics (TPHOLs 2009) (Lecture Notes in Computer Science, Vol. 5674)*. 73–78. doi:10.1007/978-3-642-03359-9_6

[18] Jacques Carette, Chris Heunen, Robin Kaarsgaard, and Amr Sabry. 2024. With a few square roots, quantum computing is as easy as Π. *Proceedings of the ACM on Programming Languages* 8, POPL (2024), 29 pages. arXiv:2310.14056 doi:10.1145/3632861

[19] Christophe Chareton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. 2021. An automated deductive verification framework for circuit-building quantum programs. In *Proceedings of the 30th European Symposium on Programming (ESOP 2021) (Lecture Notes in Computer Science, Vol. 12648)*. 148–177. doi:10.1007/978-3-030-72019-3_6

[20] Vikraman Choudhury, Jacek Karwowski, and Amr Sabry. 2022. Symmetries in reversible programming: From symmetric rig groupoids to reversible programming languages. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 6 (2022), 32 pages. doi:10.1145/3498667

[21] Alexandre Clément. 2019. A complete set of relations for the real Clifford+T group. Internship report, unpublished. https://www.mathstat.dal.ca/~aclement/Case%20distinction.pdf

[22] Alexandre Clément. 2026. A complete equational theory for real-Clifford+CH quantum circuits. In *41st Annual Symposium on Logic in Computer Science (LICS 2026) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 380)*. 28:1–28:26. doi:10.4230/LIPIcs.LICS.2026.28

[23] Alexandre Clément, Noé Delorme, and Simon Perdrix. 2024. Minimal equational theories for quantum circuits. In *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2024)*. Article 27, 14 pages. arXiv:2311.07476 doi:10.1145/3661814.3662088

[24] Alexandre Clément, Noé Delorme, Simon Perdrix, and Renaud Vilmart. 2024. Quantum circuit completeness: Extensions and simplifications. In *Proceedings of the 32nd EACSL Annual Conference on Computer Science Logic (CSL 2024) (Leibniz International Proceedings in Informatics, Vol. 288)*. 20:1–20:23. arXiv:2303.03117 doi:10.4230/LIPIcs.CSL.2024.20

[25] Alexandre Clément, Nicolas Heurtel, Shane Mansfield, Simon Perdrix, and Benoît Valiron. 2023. A complete equational theory for quantum circuits. In *Proceedings of the 38th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2023)*. 1–13. arXiv:2206.10577 doi:10.1109/LICS56636.2023.10175801

[26] J. Robin B. Cockett and Cole Comfort. 2019. The category TOF. In *Proceedings of the 15th International Conference on Quantum Physics and Logic (QPL 2018) (Electronic Proceedings in Theoretical Computer Science, Vol. 287)*. 67–84. doi:10.4204/EPTCS.287.4

[27] Robin Cockett, Cole Comfort, and Priyaa Srinivasan. 2018. The category CNOT. In *Proceedings of the 14th International Conference on Quantum Physics and Logic (QPL 2017) (Electronic Proceedings in Theoretical Computer Science, Vol. 266)*. 258–293. doi:10.4204/EPTCS.266.18

[28] H. S. M. Coxeter and W. O. J. Moser. 1980. *Generators and relations for discrete groups* (4th ed.). Ergebnisse der Mathematik und ihrer Grenzgebiete, Vol. 14. Springer, Berlin. doi:10.1007/978-3-662-21943-0

[29] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 theorem prover and programming language. In *Proceedings of the 28th International Conference on Automated Deduction (CADE 2021) (Lecture Notes in Computer Science, Vol. 12699)*. 625–635. doi:10.1007/978-3-030-79876-5_37

[30] Noé Delorme and Simon Perdrix. 2026. Diagrammatic reasoning with control as a constructor, applications to quantum circuits. In *Proceedings of the 29th International Conference on Foundations of Software Science and Computation Structures (FoSSaCS 2026) (Lecture Notes in Computer Science, Vol. 16503)*. 240–261. arXiv:2508.21756 doi:10.1007/978-3-032-22730-0_12

[31] Wang Fang, Chris Heunen, and Robin Kaarsgaard. 2026. Hadamard-Π: Equational quantum programming. *Proceedings of the ACM on Programming Languages* 10, POPL, Article 5 (2026), 27 pages. arXiv:2506.06835 doi:10.1145/3776647

[32] Shiguang Feng and Lvzhou Li. 2026. A complete set of transformation rules for reversible circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 45, 8 (2026), 3711–3724. arXiv:2508.17273 doi:10.1109/TCAD.2025.3641533

[33] Nikhil Garg. 2026. EconCSLib: AI-assisted Lean formalization for economics & computation research. arXiv:2606.13306

[34] Brett Giles and Peter Selinger. 2013. Exact synthesis of multiqubit Clifford+T circuits. *Physical Review A* 87, 3 (2013), 032332. arXiv:1212.0506 doi:10.1103/PhysRevA.87.032332

[35] Andrew N. Glaudell, Neil J. Ross, and Jacob M. Taylor. 2021. Optimal two-qubit circuits for universal fault-tolerant quantum computation. *npj Quantum Information* 7 (2021), 103. arXiv:2001.05997 doi:10.1038/s41534-021-00424-z

[36] Seth Eveson Murray Greylyn. 2014. *Generators and relations for the group* $U_4(\mathbb{Z}[1/\sqrt{2}, i])$. MSc thesis. Dalhousie University. arXiv:1408.6204

[37] David Gross. 2006. Hudson's theorem for finite-dimensional quantum systems. *Journal of Mathematical Physics* 47, 12 (2006), 122107. arXiv:quant-ph/0602001 doi:10.1063/1.2393152

[38] Shamgar Gurevich and Ronny Hadani. 2007. The geometric Weil representation. *Selecta Mathematica* 13, 3 (2007), 465–481. arXiv:math/0610818 doi:10.1007/s00029-007-0047-3

[39] Thomas Hales. 2026. What mathematicians should know about the Lean theorem prover: Questions of reliability and AI. Guest post on Terence Tao's blog *What's new*, 9 October 2026. https://terrytao.wordpress.com/2026/10/09/what-mathematicians-should-know-about-the-lean-theorem-proverquestions-of-reliability-and-ai/

[40] Keming He, Jiahan Chen, Enji Xiong, Zhongyi Ni, Lei Zhang, and Xin Wang. 2026. QECert: Formal foundations for quantum error-correction theory in Lean. arXiv:2611.05047

[41] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. 2021. A verified optimizer for quantum circuits. *Proceedings of the ACM on Programming Languages* 5, POPL, Article 37 (2021), 29 pages. doi:10.1145/3434318

[42] Derek F. Holt, Bettina Eick, and Eamonn A. O'Brien. 2005. *Handbook of computational group theory*. Chapman & Hall/CRC.

[43] Kazuo Iwama, Yahiko Kambayashi, and Shigeru Yamashita. 2002. Transformation rules for designing CNOT-based quantum circuits. In *Proceedings of the 39th Design Automation Conference (DAC 2002)*. ACM, 419–424. doi:10.1145/513918.514026

[44] Donald E. Knuth and Peter B. Bendix. 1970. Simple word problems in universal algebras. In *Computational Problems in Abstract Algebra*, John Leech (Ed.). Pergamon Press, Oxford, 263–297. doi:10.1016/B978-0-08-012975-4.50028-X

[45] Yves Lafont. 2003. Towards an algebraic theory of Boolean circuits. *Journal of Pure and Applied Algebra* 184, 2–3 (2003), 257–310. doi:10.1016/S0022-4049(03)00069-0

[46] Adrian Lehmann, Ben Caldwell, Bhakti Shah, William Spencer, and Robert Rand. 2026. VyZX: Formal verification of a graphical quantum language. *ACM Transactions on Programming Languages and Systems* 48, 3 (2026), 1–39. doi:10.1145/3807780

[47] Sarah Meng Li, Michele Mosca, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2025. A complete and natural rule set for multi-qutrit Clifford circuits. In *Proceedings of the 22nd International Conference on Quantum Physics and Logic (QPL 2025) (Electronic Proceedings in Theoretical Computer Science, Vol. 426)*. 23–78. arXiv:2508.14670 doi:10.4204/EPTCS.426.2

[48] Sarah Meng Li, Neil J. Ross, and Peter Selinger. 2021. Generators and relations for the group $O_n(\mathbb{Z}[1/2])$. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical*

Computer Science, Vol. 343). 210–264. arXiv:2106.01175 doi:10.4204/EPTCS.343.11

[49] Sirui Lu, Ruixuan Deng, Yanqiao Zhu, and Zhengfeng Ji. 2026. Long-horizon autoformalization of a core theorem underlying MIP* = RE. arXiv:2609.19814

[50] Saunders Mac Lane. 1965. Categorical algebra. *Bulletin of the American Mathematical Society* 71, 1 (1965), 40–106. doi:10.1090/S0002-9904-1965-11234-4

[51] Wilhelm Magnus, Abraham Karrass, and Donald Solitar. 2004. *Combinatorial group theory: Presentations of groups in terms of generators and relations* (2nd revised ed.). Dover Publications, Mineola, NY.

[52] Justin Makary, Neil J. Ross, and Peter Selinger. 2021. Generators and relations for real stabilizer operators. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 14–36. arXiv:2109.05655 doi:10.4204/EPTCS.343.2

[53] Dmitri Maslov, Gerhard W. Dueck, D. Michael Miller, and Camille Negrevergne. 2008. Quantum circuit simplification and level compaction. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 27, 3 (2008), 436–444. doi:10.1109/TCAD.2007.911334

[54] Yunseong Nam, Neil J. Ross, Yuan Su, Andrew M. Childs, and Dmitri Maslov. 2018. Automated optimization of large quantum circuits with continuous parameters. *npj Quantum Information* 4 (2018), 23. doi:10.1038/s41534-018-0072-4

[55] Ulf Norell. 2007. *Towards a practical programming language based on dependent type theory*. Ph. D. Dissertation. Chalmers University of Technology.

[56] Zoe Paraskevopoulou. 2026. Machine-generated, machine-checked proofs for a verified compiler (experience report). arXiv:2602.20082

[57] Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: A core language for quantum circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017)*. 846–858. doi:10.1145/3009837.3009894

[58] Yuanjie Ren, Jinzheng Li, and Yidi Qi. 2026. MerLean: An agentic framework for autoformalization in quantum computation. arXiv:2602.16554

[59] Yuanjie (Collin) Ren, Zhicheng Zhang, Junyi Liu, Jinzheng Li, Yidi Qi, and Mingsheng Ying. 2026. QEDIA: Quantum program verification in Lean. arXiv:2610.17708

[60] Issei Sakashita. 2013. A complete transformation rule set and a minimal equation set for CNOT-based 3-qubit quantum circuits (Draft). arXiv:1304.5916

[61] William Schober and Scott Wesley. 2026. A complete equational theory for quantum circuits with generalized control. In *Proceedings of the 23rd International Conference on Quantum Physics and Logic (QPL 2026) (Electronic Proceedings in Theoretical Computer Science, Vol. 456)*. 283–349. doi:10.4204/EPTCS.456.11

[62] Peter Selinger. 2015. Generators and relations for n-qubit Clifford operators. *Logical Methods in Computer Science* 11, 2, Article 10 (2015). arXiv:1310.6813 doi:10.2168/LMCS-11(2:10)2015

[63] Vivek V. Shende, Aditya K. Prasad, Igor L. Markov, and John P. Hayes. 2003. Synthesis of reversible logic circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 22, 6 (2003), 710–722. doi:10.1109/TCAD.2003.811448

[64] Sergey Shirobokov, Ben Jaderberg, Katerina Hristova, George Tsoukalas, Johannes Bausch, Eric Wieser, Francisco J. H. Heras, Nicholas C. Rubin, Guang Hao Low, Sébastien Racanière, Nenad Tomašev, Lei M. Zhang, James D. Watson, Tanuj Khattar, Aws Albarghouthi, Stavan Jain, Harun Khan, Leopoldo Sarra, Andrew W. Senior, Michael Broughton, Hussain Masoom, Sergio Boixo, Ryan Babbush, Swarat Chaudhuri, and Pushmeet Kohli. 2026. Discovering formally verified quantum programs with AI. arXiv:2611.09822

[65] Charles C. Sims. 1994. *Computation with finitely presented groups*. Encyclopedia of Mathematics and its Applications, Vol. 48. Cambridge University Press.

[66] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. 2020. t|ket⟩: A retargetable compiler for NISQ devices. *Quantum Science and Technology* 6, 1 (2020), 014003. doi:10.1088/2058-9565/ab8e92

[67] The mathlib Community. 2020. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. 367–381. doi:10.1145/3372885.3373824

[68] Mingkuan Xu, Zikun Li, Oded Padon, Sina Lin, Jessica Pointing, Auguste Hirth, Henry Ma, Jens Palsberg, Alex Aiken, Umut A. Acar, and Zhihao Jia. 2022. Quartz: Superoptimization of quantum circuits. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2022)*. 625–640. doi:10.1145/3519939.3523433

[69] Shigeru Yamashita and Igor L. Markov. 2010. Fast equivalence-checking for quantum circuits. *Quantum Information and Computation* 10, 9–10 (2010), 721–734. doi:10.26421/QIC10.9-10-1

[70] Lei Zhang, Mingrui Jing, Yusheng Zhao, Jiahong Liu, and Xin Wang. 2026. Lean-QuantumAlg: A framework for AI agents to produce quantum algorithms that humans can trust. arXiv:2610.30216

[71] Li Zhou, Gilles Barthe, Pierre-Yves Strub, Junyi Liu, and Mingsheng Ying. 2023. CoqQ: Foundational verification of quantum programs. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 29 (2023), 33 pages. doi:10.1145/3571222

A Rings, Matrices, and Quantum Gates

This appendix defines the notation of Figure 2 and gives the source of each of its matrix groups. It mostly follows Amy et al. [2], whose families are shown in the figure. Indices start at 0, as in Section 2 and [11], whereas [2, 31, 34, 48] start them at 1. We write $\mathbb{N} = \{0, 1, 2, \dots\}$, i for the imaginary unit, and $\omega = e^{i\pi/4} = (1+i)/\sqrt{2}$.

Rings. For a subring R of $\mathbb{C}$ and $\alpha_1, \dots, \alpha_s \in \mathbb{C}$, $R[\alpha_1, \dots, \alpha_s]$ is the smallest subring of $\mathbb{C}$ that contains R and $\alpha_1, \dots, \alpha_s$. The ring of *dyadic fractions* is $\mathbb{D} = \mathbb{Z}[1/2] = \{u/2^q : u \in \mathbb{Z},\ q \in \mathbb{N}\}$. Following [2, Definitions 3.1–3.3], for $\alpha \in \{\sqrt{2},\ i,\ i\sqrt{2}\}$, $\mathbb{D}[\alpha] = \{x_0 + x_1\alpha : x_0, x_1 \in \mathbb{D}\}$, and

$$\mathbb{D}[\omega] = \{x_0 + x_1\omega + x_2\omega^2 + x_3\omega^3 : x_0, \dots, x_3 \in \mathbb{D}\}, \qquad \mathbb{Z}[i] = \{x_0 + x_1 i : x_0, x_1 \in \mathbb{Z}\}.$$

Thus $\mathbb{D}[\sqrt{2}] = \mathbb{Z}[1/\sqrt{2}]$, $\mathbb{D}[i] = \mathbb{Z}[1/2, i]$, and $\mathbb{D}[\omega] = \mathbb{Z}[1/\sqrt{2}, i]$. Figure 2 uses the first of these names, and Section 2 and Table 1 the second. All of these rings are closed under complex conjugation.

Matrices and Groups. For a ring R, $R^{n\times n}$ is the set of $n \times n$ matrices with entries in R. Rows and columns are indexed by $0, \dots, n-1$, $e_0, \dots, e_{n-1}$ is the standard basis of $\mathbb{C}^n$, and I_n is the identity matrix. For a matrix M, M^T is its transpose and $M^\dagger$ its conjugate transpose. We write $\otimes$ for the Kronecker product and $\mathrm{diag}(M_1, \dots, M_r)$ for the block-diagonal matrix with diagonal blocks $M_1, \dots, M_r$. For a subring R of $\mathbb{C}$ closed under complex conjugation, the *unitary group* is $U_n(R) = \{M \in R^{n\times n} : M^\dagger M = I_n\}$. If $R \subseteq \mathbb{R}$, it equals the *orthogonal group* $O_n(R) = \{M \in R^{n\times n} : M^T M = I_n\}$. The groups of the Toffoli–Hadamard [48, Definition 5.1] and super-Gaussian [2, Equation (12)] families consist of scaled matrices,

$$L_n = \left\{M/\sqrt{2}^q \in O_n(\mathbb{R}) : M \in \mathbb{Z}^{n\times n},\ q \in \mathbb{N}\right\} \quad \text{and} \quad \left\{M/\sqrt{2}^q \in U_n(\mathbb{C}) : M \in \mathbb{Z}[i]^{n\times n},\ q \in \mathbb{N}\right\}.$$

Taking q even shows that they contain $O_n(\mathbb{D})$ and $U_n(\mathbb{D}[i])$, respectively.

Gates. A gate on k qubits is a unitary $2^k \times 2^k$ matrix, so a gate on no qubits, such as ω, is a scalar. We write $|0\rangle = e_0$ and $|1\rangle = e_1$ in $\mathbb{C}^2$, and $|x_1 \cdots x_k\rangle = |x_1\rangle \otimes \cdots \otimes |x_k\rangle$, which is e_x for the number x with binary digits $x_1 \cdots x_k$. In circuit diagrams, the first tensor factor is the top wire. The one-qubit gates are [2, Section 4]

$$\mathrm{X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \mathrm{Z} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \qquad \mathrm{S} = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \qquad \mathrm{T} = \begin{pmatrix} 1 & 0 \\ 0 & \omega \end{pmatrix},$$
$$\mathrm{H} = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad \mathrm{F} = \frac{1}{2}\begin{pmatrix} 1+i\sqrt{2} & 1 \\ 1 & -1+i\sqrt{2} \end{pmatrix} = \frac{\mathrm{H} + iI_2}{\sqrt{2}},$$

and $\omega\mathrm{H}$, the scalar ω times H. The two-qubit gate $\mathrm{H} \otimes \mathrm{H}$ is also written K [48, Definition 2.4]. For a gate G on m qubits, the *controlled* gate $\mathrm{C}G = \mathrm{diag}(I_{2^m}, G)$ on $m+1$ qubits applies G to the last m qubits if the first is $|1\rangle$. This defines CX, CZ, CS, CH, CCX = CCX, and CCZ = CCZ.

Circuits and Ancillas. In a circuit on k qubits, a gate G on m qubits applied to the distinct tensor positions $j_1, \dots, j_m \in \{1, \dots, k\}$ denotes $P(G \otimes I_{2^{k-m}})P^{-1}$. Here P is the permutation matrix with $P(v_1 \otimes \cdots \otimes v_k) = u_1 \otimes \cdots \otimes u_k$, where $u_{j_l} = v_l$ for $l \le m$ and $v_{m+1}, \dots, v_k$ fill the other positions in order. A circuit C that applies gates denoting $G_1, \dots, G_\ell$, in this order, denotes $[\![C]\!] = G_\ell \cdots G_1$. A circuit C on $k+r$ qubits *represents* $W \in U_{2^k}(\mathbb{C})$ *with* r *clean ancillas* if $[\![C]\!](v \otimes |0\rangle^{\otimes r}) = (Wv) \otimes |0\rangle^{\otimes r}$ for every $v \in \mathbb{C}^{2^k}$ [2, 34]. Without ancillas, that is, for $r = 0$, this means $[\![C]\!] = W$.

Multi-Level Matrices. Let M be an $m \times m$ matrix with entries M_{jl} for $1 \le j, l \le m$, and let $a_1, \dots, a_m \in \{0, \dots, n-1\}$ be distinct. The *m-level matrix of type* M is the $n \times n$ matrix $M_{[a_1,\dots,a_m]}$

with [2, Definition 3.7]

$$M_{[a_1,\ldots,a_m]}\, e_{a_l} = \sum_{j=1}^{m} M_{jl}\, e_{a_j} \quad (1 \le l \le m) \qquad \text{and} \qquad M_{[a_1,\ldots,a_m]}\, e_b = e_b \quad (b \notin \{a_1, \ldots, a_m\}).$$

A scalar z is a 1×1 matrix, so $z_{[a]}$ multiplies e_a by z. For example, $(-1)_{[a]}$ negates e_a, $\mathrm{X}_{[a,b]}$ exchanges e_a and e_b, and $\mathrm{H}_{[a,b]}$ sends e_a to $(e_a + e_b)/\sqrt{2}$ and e_b to $(e_a - e_b)/\sqrt{2}$. Two generators in Figure 2 are not multi-level matrices, namely the scalar matrix ωI_n and, for even n, $I_{n/2} \otimes \mathrm{H} = \mathrm{diag}(\mathrm{H}, \ldots, \mathrm{H})$, which on k qubits is H on the last qubit.

The Families. Table 2 gives the source of each matrix group of Figure 2, with its gates and generators as the source states them. The presentations of [11, 31, 48] use these generators, some with their indices restricted to increasing order, and [11] uses $(\omega\mathrm{H})^\dagger$ in place of $\omega\mathrm{H}$.

Family	Group	Gates	Generators	Source
Integral	$O_n(\mathbb{D})$	X, CX, CCX, H ⊗ H	$(-1)_{[a]}$, $\mathrm{X}_{[a,b]}$, $(\mathrm{H} \otimes \mathrm{H})_{[a,b,c,d]}$	Eq. (2), Cor. 5.6
Toffoli–Hadamard	L_n	X, CX, CCX, H	those of Integral, $I_{n/2} \otimes \mathrm{H}$	Eq. (5), Cor. 5.11
Real	$O_n(\mathbb{D}[\sqrt{2}])$	X, CX, CCX, H, CH	$(-1)_{[a]}$, $\mathrm{X}_{[a,b]}$, $\mathrm{H}_{[a,b]}$	Eq. (7), Cor. 5.16
Imaginary	$U_n(\mathbb{D}[i\sqrt{2}])$	X, CX, CCX, F	$(-1)_{[a]}$, $\mathrm{X}_{[a,b]}$, $\mathrm{F}_{[a,b]}$	Eq. (9), Cor. 5.21
Gaussian	$U_n(\mathbb{D}[i])$	X, CX, CCX, ωH, S	$i_{[a]}$, $\mathrm{X}_{[a,b]}$, $(\omega\mathrm{H})_{[a,b]}$	Eq. (10), Cor. 5.27
Super-Gaussian	unitary $M/\sqrt{2}^q$, $M \in \mathbb{Z}[i]^{n\times n}$	X, CX, CCX, H, S	those of Gaussian, ωI_n	Eq. (13), Cor. 5.31
Clifford+T	$U_n(\mathbb{D}[\omega])$	ω, H, S, CX, T	of types X, H, T, ω	[34, Thm. 1, Lem. 6]

Table 2. The sources of the matrix groups of Figure 2. Equation and corollary numbers refer to Amy et al. [2]. The cited corollary or theorem states that circuits over the gates represent, with at most one clean ancilla, exactly the elements of the group. The cited equation or lemma gives multi-level generators of the group, each with distinct indices.

Figure 2 differs from these sources in two ways, neither of which changes the matrices that circuits represent or the group that the generators generate. First, where a gate set contains H, ωH, or F, the figure lists CZ and CCZ in place of CX and CCX, and Z in place of X unless S is listed. It also lists the Clifford+T gates as {H, T, CZ}. The identities $\mathrm{X} = \mathrm{HZH} = (\omega\mathrm{H})\mathrm{Z}(\omega\mathrm{H})^\dagger = \mathrm{F}^2\mathrm{ZF}^6$, applied to the target, write each gate of one list as a circuit over the other, as do $\mathrm{Z} = \mathrm{S}^2$, $\mathrm{S} = \mathrm{T}^2$, and $\omega = (\mathrm{SH})^3$. Second, outside the integral family, the figure omits the generators $\mathrm{X}_{[a,b]}$ and $\mathrm{T}_{[a,b]}$ of the sources, which are products of those it lists. Indeed,

$$\mathrm{X}_{[a,b]} = \mathrm{H}_{[a,b]}(-1)_{[b]}\mathrm{H}_{[a,b]} = \mathrm{F}^2_{[a,b]}(-1)_{[b]}\mathrm{F}^6_{[a,b]} = (\omega\mathrm{H})_{[a,b]}\, i^2_{[b]}\, (\omega\mathrm{H})^\dagger_{[a,b]},$$

$$(-1)_{[b]} = \omega^4_{[b]}, \qquad \mathrm{T}_{[a,b]} = \omega_{[b]},$$

$$(I_{n/2} \otimes \mathrm{H})(-1)_{[2j+1]}(I_{n/2} \otimes \mathrm{H}) = \mathrm{X}_{[2j,2j+1]}, \qquad (\mathrm{H} \otimes \mathrm{H})_{[a,b,c,d]}\, \mathrm{X}_{[c,d]}\, (\mathrm{H} \otimes \mathrm{H})_{[a,b,c,d]} = \mathrm{X}_{[b,d]},$$

and in the Toffoli–Hadamard family the identities of the third line make every $\mathrm{X}_{[a,b]}$ a product of $(-1)_{[a]}$, $(\mathrm{H} \otimes \mathrm{H})_{[a,b,c,d]}$, and $I_{n/2} \otimes \mathrm{H}$.

B An Introduction to the Framework

EquaCert is built on a library for presentations, normal forms, and completeness of circuit relation sets [9]. This appendix introduces it on its smallest complete case study: circuits built from a single two-wire gate, the swap σ, which present the symmetric groups. The walkthrough exhibits the pipeline — gates, relations, cosets, tower, semantics, uniqueness, completeness, presentation — that the formalizations by normalization instantiate with richer gates and richer actions. The appendix closes with the product constructions (Appendix B.5) by which such presentations are combined.

B.1 Gates, Generators, and Circuits

A gate set is a family $\mathsf{Gate} : \mathbb{N} \to \mathsf{Set}$ giving, for each arity, the gates on that many adjacent wires; for permutations there is one gate, on two wires. The generic circuit layer of the library turns any such family into wire-indexed *generators*: a gate placed at the bottom of the wires, or a generator shifted up by one wire.

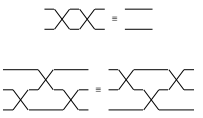
```
data Gate : ℕ → Set where
  σ-gate : Gate 2
data Gen : ℕ → Set where
  gate₁ : Gate 1 → Gen (1+ n)  - a 1-ary gate on the bottom wire
  gate₂ : Gate 2 → Gen (2+ n)  - a 2-ary gate on the bottom two wires
  _↥ : Gen n → Gen (1+ n)      - the same generator, one wire higher
```

Here $_{1+}\ n$ and $_{2+}\ n$ are the library's patterns for $1 + n$ and $2 + n$, and n is an implicit argument, so $\mathsf{gate}_2\ \sigma\text{-gate}$ is a generator at any width from two upward. EquaCert's generators also place 0-ary gates (gate_0) and three-wire gates (gate_3); the swap needs neither, so they are left out here. An n-wire *circuit* is a word of generators, $\mathsf{Circuit}\ n = \mathsf{Word}\ (\mathsf{Gen}\ n)$: a single generator $[\ g\ ]^{\mathsf{w}}$, the empty circuit ε — n bare wires, the identity circuit at that width — or a sequential composition $c \bullet c'$. We write σ for the one-letter circuit $[\ \mathsf{gate}_2\ \sigma\text{-gate}\ ]^{\mathsf{w}}$ and $c \uparrow$ for c shifted up one wire ($_\uparrow\ =\ \mathsf{wmap}\ _\!\!\updownarrow$). Read diagrammatically, with wires drawn bottom-up and a word drawn left to right as written, σ is a crossing of the bottom two wires, $\sigma \uparrow$ the same crossing one wire higher, and $\sigma \bullet \sigma \uparrow \bullet \sigma$ the three-wire braid on the left of the yang-baxter equation drawn beside its axiom below.

B.2 Relations: Two Axioms, Structure for Free

The group-specific relations are an inductive family indexed by the wire count. Each axiom is stated once, with its gates at the bottom of the wires and the width left open (the implicit n), so that one constructor covers every width at which the axiom fits; the structural rule cong↑ below moves it to any height. (Agda names may be mixfix: each underscore in _SRel,_===_ marks an argument position, so the family is applied as n SRel, w === v, read "$w = v$ is an axiom on n wires".)

```
data _SRel,_===_ : (n : ℕ) → WRel (Gen n) where
  order : (2+ n) SRel, σ • σ === ε
  yang-baxter : (3+ n) SRel, σ • σ ↑ • σ === σ ↑ • σ • σ ↑
```



These two axioms are two thirds of the Coxeter presentation of S_n: $\sigma^2 = \varepsilon$ and the braid (Yang–Baxter) relation. The third Coxeter relation, that swaps on disjoint wires commute, is not specific to permutations: it is one of the relations every circuit theory needs, whatever its gates. By a *circuit theory* we mean a gate set with gate-specific relations, presenting for each width n a monoid of n-wire circuits under sequential composition. Such circuits are the fixed-width slices of a symmetric monoidal category (a PROP), and the monoidal structure leaves exactly two traces at fixed width: the shift $c \mapsto c \uparrow$ is "tensoring with an identity wire", so equations must be preserved by it, and the

interchange law says that gates on disjoint wires commute. (EquaCert has the same commutation rule, comm_3, for three-wire gates, and admits 0-ary gates — global scalars, available at every width — with one more structural rule, that a scalar is identified with its own shift, $\omega\uparrow{=}\omega$; that it then commutes with every generator is a theorem, once scalars are assumed to commute with one another, which is vacuous for a single scalar.) These are the structural rules every circuit theory shares — what a pen-and-paper presentation leaves implicit in "a congruence compatible with composition and tensor product" — and the circuit layer adds them generically: Lift-Relation extends any gate-specific family to the full relation set _VRel,_===_:

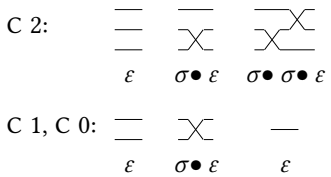
```
data _VRel,_===_ : (n : ℕ) → CRel n where
  srel  : n SRel, w === v → n VRel, w === v
  cong↑ : n VRel, w === v → (1+ n) VRel, w ↑ === v ↑
  comm₁ : (h : Gate 1) (g : Gen n) → (1+ n) VRel, [ g ↑ ]ʷ • [ gate₁ h ]ʷ === [ gate₁ h ]ʷ • [ g ↑ ]ʷ
  comm₂ : (h : Gate 2) (g : Gen n) → (2+ n) VRel, [ g ↑ ↑ ]ʷ • [ gate₂ h ]ʷ === [ gate₂ h ]ʷ • [ g ↑ ↑ ]ʷ
```

From here on, $\approx$ denotes the monoid congruence generated by n VRel,_===_: the least relation containing the axioms and closed under reflexivity, symmetry, transitivity, congruence of $\bullet$, associativity, and the unit laws; deciding it is the word problem. Because σ is its own inverse and shifts of invertible generators are invertible, the presented monoid is a group; we witness this by a Grouplike proof (every generator has a left inverse), by induction on generators.

B.3 Cosets and the Staircase Normal Form

The engine behind the normal form is coset enumeration along the chain $S_1 < S_2 < \cdots < S_n < S_{n+1}$, where S_n sits inside S_{n+1} as the permutations fixing the bottom wire — syntactically, the circuits of the form $c\uparrow$. Read as permutations, two circuits lie in the same right coset of S_n exactly when they bring the same wire to the bottom position; there are $n{+}1$ choices, so S_n has index $n{+}1$, and a natural representative of each coset is the *descending staircase* of swaps that carries that wire down. In the library a coset is not a set of circuits but a label naming one, an element of the finite type C n; here the label is the length of the staircase, as a unary numeral bounded by the width, and the *section* $[\ _\ ]^{\mathrm{c}}$ maps it to the staircase circuit. The representatives on three, two, and one wires are drawn on the right:

```
data C : ℕ → Set where
  ε : C n
  σ•_ : C n → C (1+ n)
[_]ᶜ : C n → Circuit (1+ n)
[ ε ]ᶜ      = ε
[ σ• c ]ᶜ = σ • ([ c ]ᶜ ↑)
```

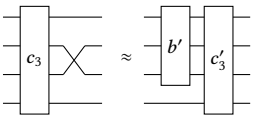


So C n has exactly the $n{+}1$ elements drawn above for $n = 2, 1, 0$, matching the index of S_n in S_{n+1}. Why this yields a normal form is the classical argument of computational group theory [42, 65]: every $f \in S_{n+1}$ lies in one coset, say with representative c_n, so $f = f' \cdot c_n$ with $f' \in S_n$, and recursing on f' writes f uniquely as $c_1 c_2 \cdots c_n$ with $c_k \in \mathrm{C}\ k$ — a numeral in the *factorial number system*, one digit per level, since $|S_{n+1}| = (n{+}1) \cdot n \cdots 2$.

Computing this factorization is the job of the coset table. Given a coset, represented by its staircase, and one generator, the table says where their product lands — the updated coset — and what is left over on the wires above — the residual circuit — so that staircase followed by generator equals residual (shifted up) followed by updated staircase. Pushing σ into a staircase can lengthen it, cancel with it ($\sigma^2 = \varepsilon$), or pass it through and shift it up (Yang–Baxter), or commute past; for a generator that does not touch the bottom wire we can recurse. The residual is empty or a single

swap here, but in richer gate sets it can be a longer circuit, which is why the table returns a circuit on one wire fewer rather than a generator (Figure 13 draws the four cases):

```
ract : C (1+ n) → Gen (2+ n) → Circuit (1+ n) × C (1+ n)
ract ε σ-gen = ε , σ• ε                  - grows
ract (σ• ε) σ-gen = ε , ε                - cancels (σ² = ε)
ract (σ• σ• c) σ-gen = σ , σ• σ• c       - passes (Yang-Baxter)
ract ε (g ↥) = [ g ]ʷ , ε                - commutes past
ract (σ• c) (g ↥) = proj₁ (ract c g) ↑ , σ• (proj₂ (ract c g))   - recurses
```



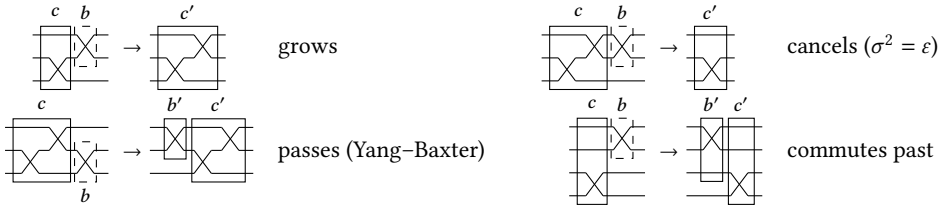


Fig. 13. Pushing a swap through a staircase: the four cases of the coset table ract as circuit rewrites.

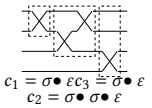
σ-gen abbreviates gate$_2$ σ-gate. The last clause is the recursion: a shifted generator meets a staircase by meeting its tail one wire down. Each clause is certified by the *soundness law*, pictured on the right and proved once per table by a short equational derivation from the axioms:

```
ract-sound : ∀ c b → let (b' , c') = ract c b in [ c ]ᶜ • [ b ]ʷ ≈ b' ↑ • [ c' ]ᶜ
```

The stateful word traversal _^{t} extends ract to whole circuits, threading the coset from left to right and concatenating the residuals; run from the identity coset, it maps an $(n{+}2)$-wire circuit to an $(n{+}1)$-wire circuit and a coset — one level of normalization.

Iterating the level is the coset tower, whose carrier grows by one digit per level:

```
NF : ℕ → Set
NF 0 = ⊤
NF (suc k) = NF k × C k - ⊤ × C 0 × C 1 × ··· × C k
```



so NF n has exactly $n!$ elements. The normal-form function nf : Circuit $n \longrightarrow$ NF n runs the tables; its *section* inv-nf : NF $n \longrightarrow$ Circuit n rebuilds a circuit from its digits by concatenating the staircases, suitably shifted, as drawn on the right for $(\mathsf{tt}, c_0, c_1, c_2, c_3) \in \mathsf{NF}\ 4$, the digit $c_0 \in \mathsf{C}\ 0$ being the empty staircase — one staircase per level, each shifted up to the wires of its level: a "staircase of staircases".

B.4 Semantics, Uniqueness, and Completeness

So far everything was syntax. The development interprets circuits in the group Permutation′-group n of permutations of Fin n, with σ swapping the bottom two elements. *Soundness* — $w \approx v$ implies $[\![w]\!] \doteq [\![v]\!]$ — is a small case analysis over the axioms. The heart of the matter is the *unique normal form* property, that normal forms with equal denotations are equal: $[\![\ \mathsf{inv\text{-}nf}\ u\ ]\!] \doteq [\![\ \mathsf{inv\text{-}nf}\ v\ ]\!]$ implies $u \equiv v$. It is proved by induction on the tower: the lower digits fix the last point, so the top digit alone determines where the permutation sends it; equal denotations therefore force equal top digits, and cancelling them reduces the claim to the level below. Soundness and uniqueness are exactly the premises of the generic lemma by-normalization, which concludes *completeness*: $[\![w]\!] \doteq [\![v]\!]$ implies $w \approx v$. Together with surjectivity (every permutation is a product of adjacent transpositions), these assemble into the library's *presentation record*: a Grouplike witness, which

makes the presented monoid a group, and a proof that [[_]] is a group isomorphism from that group onto the target (IsGroupIsomorphism is the standard library's, between the group the witness induces and G; the record's module plumbing that says so is elided):

```
record _IsPresentationOf_ (_===_ : WRel X) (G : Group a ℓ) : Set (a ⊔ ℓ) where
  field
    gl : Grouplike _===_
    [[_]] : Word X → Group.Carrier G
    iso : IsGroupIsomorphism [[_]]
```

The presentation theorems of Section 3 have the shape of this one, or that of IsSubPresentationOf, which asks for a group monomorphism in place of the isomorphism:

```
symmetric-presentation : ∀ n → (n VRel,_===_) IsPresentationOf (Permutation′-group n)
```

read: the monoid of swap circuits modulo {order, yang-baxter} and the structural rules *is* the group of permutations of Fin n.

B.5 Products of Presentations, and Their Presentation Theorems

Presentations such as the one just obtained can be combined. Relation sets over a disjoint union of alphabets are built from a single primitive, a three-way join whose *mixed* component is the design parameter:

```
data _◇_◇_ (Γ₁ : WRel A) (Γ₂ : WRel B) (Γ₃ : WRel (A ⊎ B)) : WRel (A ⊎ B) where
  left  : Γ₁ u v → (Γ₁ ◇ Γ₂ ◇ Γ₃) [ u ]ₗ [ v ]ₗ
  right : Γ₂ u v → (Γ₁ ◇ Γ₂ ◇ Γ₃) [ u ]ᵣ [ v ]ᵣ
  mid   : Γ₃ u v → (Γ₁ ◇ Γ₂ ◇ Γ₃) u v
```

Choosing the mixed relation recovers the classical constructions: free product (EmptyRel), direct product (CommRel), semidirect product (ConjRel^{w} *conj*: moving a letter h of the acting group past a letter n of the normal subgroup conjugates n, by a *word*, $h \bullet n = conj\ h\ n \bullet h$) and definitional extension (SugarRel). For each, the library lifts normal forms from the factors to the construction and proves that if the factors present concrete groups, the constructed relation set presents the constructed group. Beyond presentations of the factors, the semidirect theorem needs two well-definedness conditions on the action: that it respects the acting group's axioms in its first argument (conj-hyph) and the normal subgroup's in its second (conj-hypn):

```
module _ (conj-hyph : ∀ {c d} n → c ===₂ d → (conj ʰ′) c n ≈₁ (conj ʰ′) d n)
         (conj-hypn : ∀ c {w v} → w ===₁ v → (conj ⁿ′) c w ≈₁ (conj ⁿ′) c v) where
  module Presentation (G1 G2 : Group 0ℓ 0ℓ)
    (p1 : Γ IsPresentationOf G1) (p2 : Δ IsPresentationOf G2) where
    dpres : (Γ ◇ Δ ◇ ConjRel^w conj) IsPresentationOf G1⋊G2
```

Here G1⋊G2 is the semidirect product of the two *semantic* groups, built from an Action record whose action is the syntactic conj transported through the two presentation isomorphisms. A further module proves the classical recipe for presenting a group *extension* $1 \to N \to G \to Q \to 1$ from presentations of N and Q, how G conjugates N, and the correction in N by which each lifted relator of Q fails to hold (cf. Proposition 2.55 of [42]): a second mixed component, RelTwist, twists each relator of the quotient by its correction, and the relation set of the recipe is

```
extension-presentation S R conj corr = S ◇ EmptyRel ◇ (ConjRel^w conj ∪ RelTwist R corr)
```

Semidirect products are the special case with trivial corrections. With these theorems a presentation of a product or of an extension is *assembled* from presentations of its factors, with no fresh coset enumeration at all.

C EquaCert Implementation Details

This appendix defines PROPs, explains the theorem notation used by the framework in Section 3, and gives the size and provenance of each formalization. Wire placement and matrix equality checking are described in Sections 3 and 3.2.

PROPs. A *PRO* is a strict monoidal category whose objects are the natural numbers and whose tensor product is addition on objects, and a *PROP* is a PRO that is moreover symmetric monoidal [50]. Circuits form a PRO [45], in which an object n is a bundle of n wires, a morphism from m to n is a circuit with m input wires and n output wires, composition places circuits in sequence, and the tensor product places them in parallel. In a PROP, every permutation of the wires is also a morphism. These permutations are generated by the swaps σ_d of the wires d and $d+1$, subject to two laws. The swap is an involution, $\sigma_d\sigma_d = \mathrm{id}$, and it is natural, so that a gate followed by a crossing of wires equals the crossing followed by the same gate on the wires to which the crossing carries it. Naturality at the swap itself is the Yang–Baxter equation $\sigma_0\sigma_1\sigma_0 = \sigma_1\sigma_0\sigma_1$. These laws let a rule stated on its own wires be applied to any wires of a larger circuit, by bringing those wires together with swaps and moving them back afterwards. Several recent completeness results for quantum circuits are stated in PROPs [14, 16].

C.1 Theorem Notation

The signatures in Section 3 use `n CRel._===_` for relations on circuits with n wires and `n MRel._===_` for relations on words over multi-level matrices of dimension n. In Statement (1), the former contains R1–R16 of Figure 1 of [52], lifted to n wires. In Statement (2), the latter contains the relations of Figure 6 of Hadamard-Π [31].

The judgement `⊢ w ≈ v` states that w and v are equal by the relations. The qubit formalizations interpret matrices in $\mathbb{D}[\omega]$ and use `_≈ᵁ_` for their equality; the qudit formalizations compare interpretations in their own groups. The `IsSubPresentationOf` record contains the semantics and its soundness and completeness proofs, and `IsPresentationOf` adds surjectivity.

Minimality is stated for the rule set `n ⊢_===_` in Statement (5). Unlike the generated equivalence relation, this retains the individual rules, so removing one is defined. The predicate `IsMinimal` therefore takes this rule set rather than its generated equivalence relation.

C.2 Size and Provenance of the Formalizations

Table 3 gives, for each row of Table 1, the origin of its Agda proof, its size, and the size of what a reader must read to know what its theorem says.

What a Reader Must Read. For each row we followed the names in the statement of its main theorem to their definitions: the relations, the gates or generators, the target group, and the semantics down to the matrix of each generator. By our count these statements and definitions take 8,207 of the 376,627 code lines, or 2.2%. Of these, 7,087 lines are in the rows' own modules (the Statement column), and 1,120 are shared by several rows: 200 lines of the framework's definitions of words, circuits, structural rules, and presentations, and 920 lines that define the rings, matrices, and matrix groups in which several rows are interpreted. Where we did not cut a module down to the definitions that a statement mentions, we counted it whole, so the counts lean high. The count for the qudit Clifford row is the least precise. Its target group is built from a presentation, and the count includes the rule set and the model of that presentation but not the general constructions of central extensions through which the group is defined. In three rows, two-qubit Clifford+T, three-qubit Clifford+CS, and Toffoli–Hadamard, the matrix of a gate is defined through a translation of the gate into multi-level generators, so this translation is among the definitions to read. The counts

	Source	Proof	Code lines	Generated	Ported	Statement	Uses rows
1	Greylyn [2014]	S	7,348	0	0	91	
2	Bian and Selinger [2021]	S	7,754	0	0	160	
3	Li et al. [2021]	S	13,684	1,712	0	450	
4	Fang et al. [2026]	S	9,798	2,087	0	114	5
5	Clément [2019]	S	9,167	1,948	0	114	4
6	Selinger [2015]	S	10,768	6,855	0	228	
7	Bian, Li et al. [2026]	E	57,867	224	0	2,167	
8	Makary et al. [2021]	S	11,081	5,957	0	221	
9	Amy et al. [2018]	S	5,064	82	0	235	
10	Blake [2026b]	S	10,821	140	0	530	
11	Bian and Selinger [2023a]	P	11,069	24	10,377	355	1
12	Bian and Selinger [2023b]	P	56,295	24	55,494	556	2
13	Clément [2026]	S	60,205	1,601	0	283	5
14	Amy et al. [2024]	R	28,629	22,572	0	713	3
15	Li et al. [2025]	S	53,882	49,388	0	232	
16	Blake [2026c]	S	23,195	14,469	0	638	1, 2, 6–9, 11, 12
	Total		376,627	107,083	65,871	7,087	

Table 3. Size and provenance of the Agda formalizations, row by row in the order of Table 1. The Proof column is that of Table 1. Code lines exclude blank lines and comments. Generated and Ported count the modules whose header says that a script wrote them or that they are ported from the Agda code of the source's authors. Statement counts the code lines, in the row's own modules, of the statement of the main theorem and of the definitions that it mentions. The last column lists the rows whose theorems the proof uses; definitions and general lemmas that the folder of one row supplies to others, such as rings, matrices, matrix groups, and the evaluation of matrices, are not counted.

leave out the definitions of the rings $\mathbb{D}$, $\mathbb{D}[i]$, $\mathbb{D}[\sqrt{2}]$, and $\mathbb{D}[\omega]$ and of matrices over them, which come from a separate library for rings and exact synthesis.

Origin of the Proofs. A module counts as generated or ported only when its header says so in words that our count recognizes. About 3,600 lines are in modules whose headers say so in other words, and are counted as written: derivations found by search in rows 3 and 9, and a module of arithmetic modulo 3 in row 15. So the Generated column is a lower bound. Rows 4 and 5 present the same group by two sets of relations and share lemmas: row 5 proves the case that the proof in the source of row 4 leaves open (Section 5), and uses the lemmas of row 4 on levels and its proof that the two sets of relations are equivalent. Rows 11 and 12 obtain completeness from the presentations of rows 1 and 2. Row 13 takes the completeness theorem that its source imports from row 5. Row 14 translates circuits into the groups of row 3. Row 16 transfers completeness from the rows listed. We count the commits to a row's folders under their current and earlier names. Every commit to fourteen of the sixteen rows carries the trailer of a coding agent. Of the 482 commits to the qudit Clifford row, 121 do; most of the others date from before the workflow of Section 3.2. Of the 28 commits to the Toffoli–Hadamard row, 15 do, and the other 13 are the work of the second coding agent (Section 3.2).

D Details of the Agda and Lean Comparison

This appendix gives the detailed comparison and measurement methods for the two formalizations introduced in Section 4.1.

The Lean framework is not an independent implementation. The coding agents that wrote the Lean formalizations were instructed not to consult the Agda implementation, but the Lean framework is nonetheless a translation of the Agda one. Its words, circuit generators, structural rules, and presentation and sub-presentation predicates are those of the Agda framework, down to the names of the definitions and the wording of their documentation. The two formalizations therefore share the design of their framework, and would share any mistake in it, so their agreement is weaker evidence than two independent formalizations would give. They use different representations and sometimes different proofs, so the theorem statements of each must be checked against the source papers separately. We compare their definitions, proof formalization, verification time, and execution time; Table 4 summarizes the comparison. These observations concern our two formalizations, not Agda and Lean in general.

Definitions and Semantics. The Lean framework keeps the definitions of the Agda one but adapts their representation to Mathlib. Agda uses its own word type, with explicit brackets, and the setoid-based algebra of the standard library. A presentation is a record containing a semantics and a group isomorphism, or a monomorphism for a sub-presentation. Lean uses Mathlib's free monoid and the monoid congruence generated by the relations. Its presentations are predicates in `Prop`, from which it derives isomorphisms out of the quotient. In both, the circuit infrastructure, the structural rules, and general completeness arguments are shared between formalizations.

The semantic targets also differ. Most Agda formalizations work directly with exact representations of matrix entries. Lean states its matrix groups over Mathlib's real or complex numbers and proves bridges to them from computable exact rings. In particular, the Agda qudit Clifford formalization presents a constructed algebraic model, whereas Lean proves soundness and completeness against complex matrices for every odd prime. A common source therefore does not imply identical statements or intermediate lemmas.

Proof Development. Excluding blank lines and comments, the Agda formalization of Table 1 has about 377,000 lines and the Lean formalization about 339,000 (Table 4). Figure 7(a) compares them row by row, except that for Toffoli–Hadamard it shows another pair, one proof in both assistants (see below). Lean's is shorter in thirteen of the sixteen rows, by a factor of five or more for qudit Clifford and qutrit Clifford, where the two take different routes. It is longer for Hadamard-Π, three-qubit Clifford+CS, and real Clifford+CH. Both counts include the derivations and certificates that scripts generate. The Agda proofs are explicit terms, built by pattern matching and equational reasoning, with reflection-based solvers for associativity and ring identities. Its library develops the group theory the proofs need, in about 13,200 lines against 1,200 in Lean. The Lean proofs rely on Mathlib's algebra and on tactics, with about 69,000 calls to automation such as `simp`, `omega`, `decide`, and `ring`.

Both implementations reuse normal-form, exact-synthesis, and translation arguments. Both also check generated derivations with a replay procedure that is proved correct. The proof assistant checks the computation and the theorem that a successful replay gives a derivation, so the scripts that search for derivations need not be trusted. For Clifford+CS, Lean constructs its own coset certificates. The trust checks reflect the different foundations. Every Agda module is `--safe` and uses no postulates, so the Agda formalization is constructive. The Lean formalization uses no `native_decide`, and its main theorems depend on no unfinished proof. An audit of 105 declarations reports only the axioms `propext`, `Classical.choice`, and `Quot.sound`.

Verification Time. Table 1 and Figure 7(b) give the check times, both measured on the same machine. In total Agda took 2,399 seconds and Lean 9,305, or 1,979 and 3,826 without real Clifford+CH, whose Lean build alone took 5,479 seconds, more than half of the total. Lean took between 1.2 and

	Agda	Lean
Proof development		
Logic	constructive: every module `--safe`, no postulates	classical: every main theorem uses `Classical.choice`
Library	standard library (104 modules imported), a library for rings and exact synthesis	Mathlib (56 modules imported directly)
Proofs	terms, by pattern matching and equational reasoning; reflection-based solvers	tactics: about 74,000 tactic blocks, 69,000 automation calls
Reused from the sources	the authors' Agda proofs for Clifford+T and Clifford+CS, and their derivations for Toffoli–Hadamard	no proofs; the authors' coset-action data for Clifford+T
Modules	1,509	793
Code lines	376,627	338,570
Verification time		
Decided equalities	a Boolean reduced to `true`	`decide +kernel`
Check time, sixteen rows	2,399 s	9,305 s
Faster for	thirteen rows	three rows
Execution time		
Normal forms	executable functions	existence theorems or `noncomputable` definitions (460 declared)
Benchmarked procedure	matrix of each circuit, by products of stored matrices	simulation of each circuit on every basis vector
Compiled through	GHC (Haskell)	C
Per decision, Clifford, 100 gates	5.0 ms on 2 qubits, 24 s on 6	5.8 ms on 2 qubits, 0.61 s on 6

Table 4. The Agda and Lean formalizations compared. Counts are over the files under version control in the sixteen formalization folders of Table 1, without the shared libraries, and code lines exclude blank lines and comments. Check times are those of Table 1, and execution times those of Figure 14.

4.8 times as long as Agda on twelve rows, and 13 times as long on real Clifford+CH. Its five-wire certificate modules `Family267` and `Family268` alone took about 1,741 and 1,369 seconds. Lean was faster for three rows, Clément's presentation, qudit Clifford, and qutrit Clifford, whose Lean proofs take different routes. Both assistants decide equalities of matrices and certificates by evaluation in the checker. Agda reduces a Boolean to `true`, and Lean uses `decide +kernel`.

Both columns were measured under WSL on an Intel Core i5-13400TEF with sixteen logical CPUs, about 26 GiB of RAM, and 64 GiB of swap. The Agda column comes from a check from scratch on 8 November 2026, with Agda 2.8.0 and version 2.4 of its standard library. We removed the repository's interface files, kept those of the standard library and of the library for rings and exact synthesis, and checked the shared library and then each row in an Agda process of its own,

through a module that imports every module of the row. Agda's profiler reports the time spent on each module without its imports, and a row's time is the sum over its own modules, wherever they were checked. It leaves out process startup and the loading of interface files: the whole check, shared library included, took 3,210 seconds of wall time, and no process used more than 5.1 GiB. No other proof assistant ran during the check. The Lean column comes from a clean build on 5–6 November 2026, with Lean 4.30.0 and the pinned Mathlib revision. We removed EquaCert's build artifacts, kept the Mathlib and other dependency caches, and built the folders one at a time in dependency order with `LEAN_NUM_THREADS=2`. Each time is the elapsed wall time of a folder, including Lake startup and artifact generation, and a shared dependency is counted only in the folder that defines it. The two columns therefore measure different things, per-module checking time in Agda and folder build time in Lean. Proof constructions, module boundaries, parallelism, and artifact generation also differ, so the differences cannot be attributed to the proof assistants alone. We have yet to separate the time Lean spends running automation from the time it spends checking the resulting proofs.

The build covered 810 library modules, excluding three generator templates, and took **9,415.37 seconds (2 h 36 min 55 s)** in timed build commands. Beyond the sixteen formalization folders, the shared `Circuit`, `Exact`, `Matrix`, and `NumberTheory` folders took 17.91, 15.51, 6.88, and 15.29 seconds, and the five root modules, including the audit, took 54.95 seconds. Lean builds of another project ran on the same machine during part of the run. We therefore re-timed the twenty folders and root modules they overlapped and report the second timing, during which the other project used about a quarter of one CPU on average. The two runs total 9,604 and 9,415 seconds, and the two timings of each formalization folder agree within 15%, except for Toffoli–Hadamard (178 and 135 seconds).

One Proof in Both Assistants. For Toffoli–Hadamard, the formalizations of Table 1 have 28,629 lines in Agda and 4,401 in Lean, and they prove the result by different routes. The Agda proof replays the derivations in the authors' own files as generated certificates, which make up 22,572 of its lines. The Lean proof was written from the paper, over an alphabet of macro gates, with a Knuth–Bendix system for the symmetric group S_8 that the kernel checks. To compare the two assistants on one proof, Figure 7(a) shows another pair instead (∗). A coding agent wrote a second Agda proof of the result, which finds its circuit derivations by a breadth-first search over the relations of Figure 4 of [3], and a coding agent translated it to Lean module for module, keeping its definitions, derivations, and proof route; this time we told the agent to follow the Agda. A coding agent also began translating the first Agda proof in the same way, porting its scripts so that they write the certificates in Lean, and we stopped it after 46 of the 203 modules: the relations, the 218 derivations replayed from the authors' files, the 642 certificates of the permutation fragment, and the frames built on them, 17,862 of the Agda proof's lines. On one proof the two assistants need about the same amount of text: 182,301 non-blank characters outside comments in Agda and 191,589 in Lean for the second proof and its translation, and 1,056,950 and 1,030,864 for the translated modules of the first. In lines the translation of the second proof is shorter, 5,672 against 7,161, only because its generated derivations put several rewrite steps on a line where Agda's put one, while the translated modules of the first, whose certificates take a step per line in both, are 6% longer, 18,946 against 17,862. The part of the second proof's translation that is not generated is the larger of its pair, mostly because it proves the facts about S_8 that the Agda proof imports from the shared library. Lean took 164 seconds to check it and Agda 48, 3.4 times as long, while on the two proofs of Table 1, timed in the same way, Lean took 118 seconds and Agda 111, 1.1 times as long (Table 1, timing Lean's whole build, has 135 and 111). So the pair that Table 1 and Figure 7(b) report reflects the difference between the proofs more than the difference between the assistants.

The new result of Section 6 gives a third pair. Coding agents translated the 38 Agda modules of its proof [4] to Lean module for module, again told to follow the Agda, and ported the scripts that write its generated derivations, so that they write Lean from the same search results. Sixteen further Lean modules translate what the Agda imports from the shared library: Lafont's presentation of the alternating groups with his canonical forms, the Coxeter presentation of the symmetric groups, the swap rules, and the morphisms between presentations. Again the two need about the same amount of text: 462,926 non-blank characters outside comments in Agda and 536,855 in Lean, or 484,418 without the sixteen library modules, in 24,062 and 23,083 lines. Lean took 288 seconds to check the translation and Agda 28, ten times as long, but most of Lean's time does not go to the proofs. Every Lean module first loads Mathlib, which takes about four seconds for a module with no content, and the generated derivations are fourteen modules that import one another in a chain, as in the Agda. The 54 modules took 354 seconds in all, some of them at the same time, so about 220 of these went to loading, and what is left is between four and five times Agda's time. Reaching that needed two departures from the Agda, both forced by the cost of evaluation in Lean's kernel. Applying one gate to one state, as the Agda semantics defines it, costs the kernel about half a millisecond, so the translation decides equalities of circuits on states encoded as numbers, after checking the arithmetic of each gate against the semantics once; this took its semantics module from 79 seconds to 10. And where each step of an Agda derivation carries its own evidence, which Agda fills in by evaluation, the translation evaluates the evidence of all the steps of an equation in one kernel check, 788 checks for the 15,607 steps on four wires. These six times were measured on 10 November 2026 on the machine of Table 1, by its method, with no other check running.

Execution Time. The Agda normal forms are executable functions. The Lean formalizations obtain normal forms either as existence theorems or as `noncomputable` definitions. For example, the Lean proofs for the Clifford and real stabilizer circuits of [52, 62] show that every circuit equals a normal form without defining a function that computes it, and the exact synthesis of the two-level groups is noncomputable. In all, 460 definitions in the sixteen Lean formalizations are declared `noncomputable`, so there is no Lean normalizer to time. Both formalizations can, however, execute the procedure that decides whether two circuits have the same matrix, which their soundness proofs run. We compiled this procedure for the Clifford circuits of [62] to native code, through Agda's GHC backend and Lean's C backend. We then timed both on an Intel Core i5-14400, on the same random circuits of 100 gates over H, S, and CZ, deciding that each circuit w has the matrix of $w\,\mathrm{S}^4$ (Figure 14). On two qubits Agda is slightly faster, at 5.0 against 5.8 milliseconds per decision. From three qubits on Lean is faster, by a factor of 2.1 on three qubits and 40 on six, where Agda takes 24 seconds and Lean 0.61. The gap comes mostly from the procedures rather than the compilers. Agda computes each circuit's matrix as a product of stored $2^n \times 2^n$ matrices, so its work grows about eightfold with each qubit. Lean simulates the circuit on each basis vector, so its work grows about fourfold.

E Proofs for Four-Bit Reversible Circuits

This appendix proves Theorems 6.1 and 6.2 as the Agda formalization [4] does. Each subsection names the modules of `Examples/Groups/Alt16-RevCir` that it follows, and each statement names its Agda counterpart.

E.1 Conventions and Outline

The gates and the theory are the modules Syntactics and Theory, and the proof engine is Engine. Wires are numbered from 0 at the bottom. A state of k wires is a bit string $b_{k-1} \cdots b_0$, the bit of the top wire first. Circuits are words in diagram order, and products of permutations follow the same

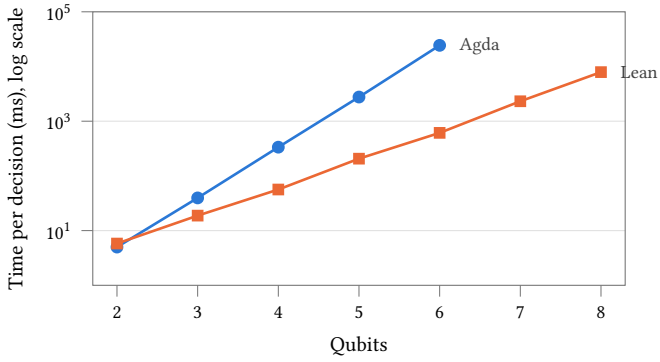


Fig. 14. The time each compiled formalization takes to decide that two Clifford circuits of 100 gates have the same matrix, by the procedure its soundness proof uses, as medians over three runs of five random circuits. The two procedures differ, and this measurement was made on a second machine.

order, so uw applies u and then w. We write $u \approx w$ when u and w are equal modulo $\mathcal{T}$, ε for the empty circuit, and G_k for the circuits on k wires modulo $\mathcal{T}$, with the product of circuits.

The gate X acts on wire 0, CX has its control on wire 1 and its target on wire 0, CCX has its controls on wires 2 and 1 and its target on wire 0, and the structural swap σ exchanges the wires 0 and 1. The *generators* of the circuits on k wires are these four gates, shifted up by any number of wires. A gate in any other position is a generator between swaps, and an open control is a filled one between two X gates on its wire. We write X_t, $\mathrm{CX}_{c\to t}$, and $\mathrm{CCX}_{ab\to t}$ for the gates with target t and controls c, or a and b in either order, with a bar over an open control as in $\mathrm{CCX}_{\bar{3}2\to 0}$, and σ_d for the swap of the wires d and $d+1$. On four wires there are twelve generators: $\mathrm{X}_0, \ldots, \mathrm{X}_3$, $\mathrm{CX}_{1\to 0}$, $\mathrm{CX}_{2\to 1}$, $\mathrm{CX}_{3\to 2}$, $\mathrm{CCX}_{21\to 0}$, $\mathrm{CCX}_{32\to 1}$, and σ_0, σ_1, σ_2. In this notation the equations of Figure 9 are

(T1) $\mathrm{X}_0\,\mathrm{X}_0 = \varepsilon$,
(T2) $\mathrm{CX}_{1\to 0}\,\mathrm{X}_1\,\mathrm{CX}_{1\to 0}\,\mathrm{X}_1 = \mathrm{X}_0$,
(T3) $\mathrm{CX}_{1\to 0}\,\mathrm{CX}_{0\to 1}\,\mathrm{CX}_{1\to 0} = \sigma_0$,
(T4) $\mathrm{CCX}_{21\to 0}\,\mathrm{X}_2\,\mathrm{CCX}_{21\to 0}\,\mathrm{X}_2 = \mathrm{CX}_{1\to 0}$,
(T5) $\sigma_1\,\mathrm{CCX}_{21\to 0}\,\sigma_1 = \mathrm{CCX}_{21\to 0}$,
(T6) $\mathrm{CX}_{0\to 1}\,\mathrm{CCX}_{21\to 0}\,\mathrm{CX}_{0\to 1} = \mathrm{CCX}_{20\to 1}\,\mathrm{CCX}_{21\to 0}\,\mathrm{CCX}_{20\to 1}$,
(T7) $\mathrm{CCX}_{32\to 1}\,\mathrm{CX}_{1\to 0}\,\mathrm{CCX}_{32\to 1} = \mathrm{CCX}_{32\to 0}\,\mathrm{CX}_{1\to 0}$,

where $\mathrm{CX}_{0\to 1} = \sigma_0\,\mathrm{CX}_{1\to 0}\,\sigma_0$, $\mathrm{CCX}_{20\to 1} = \sigma_0\,\mathrm{CCX}_{21\to 0}\,\sigma_0$, and $\mathrm{CCX}_{32\to 0} = \sigma_0\,\mathrm{CCX}_{32\to 1}\,\sigma_0$.

The proof engine and its generated derivations are described in Section 6.3.

The step between two presentations. The proof uses the following form of the Reidemeister–Schreier method several times, as StarGroupIsomorphism of the framework's Presentation.Morphism. It is the case of the method in which the subgroup is the whole group, so there is a single coset and no transversal to choose. What remains is the standard criterion that two presentations are related by Tietze transformations [51]: mutually inverse substitutions of generators that respect the relations. Let $\langle A \mid R\rangle$ and $\langle B \mid Q\rangle$ be two presentations, and let f send each generator in A to a word over B and g each generator in B to a word over A. Both extend to words, letter by letter. Suppose that

(1) for every relation $l = r$ of R, the words $f(l)$ and $f(r)$ are equal modulo Q;
(2) for every relation $l = r$ of Q, the words $g(l)$ and $g(r)$ are equal modulo R;
(3) every generator a in A is equal to $g(f(a))$ modulo R;

(4) every generator b in B is equal to $f(g(b))$ modulo Q.

If moreover every generator has an inverse word on each side, then f induces an isomorphism from the group of $\langle A \mid R\rangle$ onto that of $\langle B \mid Q\rangle$, with the inverse induced by g. We call (1)–(4) the *four facts* of the step.

The proof follows the five stages outlined in Section 6.3. The following subsections give their detailed arguments.

E.2 Soundness

This part follows the modules Semantics and Theory.

Proposition E.1 (T-sound). *On any number of wires, if $u \approx w$ then $[\![u]\!] = [\![w]\!]$.*

Proof. The two sides of each of (T1)–(T7) denote the same function at the width at which the equation is drawn, which is decided by evaluation on all states of that width. The structural rules hold at every width, by computation on the bits of the wires that they touch. The claim follows by induction on the derivation. □

E.3 Lafont's Presentation and the Cycles

This part follows the modules Numbering and Syntactics, and the library's Alternating. As in Section 6, $\omega_0, \ldots, \omega_{N-3}$ are the cycles of three consecutive points among N points. Lafont's relations between these cycles are

$$\begin{array}{ll} \text{(order)}\ \ \omega_i\,\omega_i\,\omega_i = 1, & \text{(near)}\ \ \omega_i\,\omega_{i+1}\,\omega_i = \omega_{i+1}\,\omega_{i+1}, \\ \text{(far)}\ \ \omega_i\,\omega_{i+2}\,\omega_i = \omega_{i+2}\,\omega_i\,\omega_{i+1}, & \text{(apart)}\ \ \omega_j\,\omega_i = \omega_i\,\omega_j \quad (j \geq i+3), \end{array}$$

with products in diagram order. The first three are the relations of his Theorem 4 as he draws them, and (apart) is the commutation rule of his diagrams. In formulas he writes products in the order of composition, so that the right side of (far) reads $\omega_{i+1}\,\omega_i\,\omega_{i+2}$ there, and he replaces (near) by $(\omega_i\,\omega_{i+1})^2 = 1$, which is equivalent to it given (order). The library of EquaCert proves Theorem 4 of [45]: two words in the ω_i with the same permutation are equal modulo these relations, and every even permutation of the N points is that of a word. In the library the ω_i are one three-wire gate shifted up, and (apart) is a structural rule. For $N = 16$ there are 14 cycles and $14 + 13 + 12 + 66 = 105$ relations.

Positions. The sixteen states of four wires are numbered along the reflected Gray code, in which wire 0 changes most often. Position i is the state $b_3b_2b_1b_0$ with the binary digits of $i \oplus \lfloor i/2 \rfloor$:

0	0000	4	0110	8	1100	12	1010
1	0001	5	0111	9	1101	13	1011
2	0011	6	0101	10	1111	14	1001
3	0010	7	0100	11	1110	15	1000

Two consecutive positions differ on one wire. So the positions i, $i+1$, $i+2$ are three corners of a face of the 4-cube, on which two wires change and the other two hold fixed values.

The cycles. Let $K = (\sigma_0\,\sigma_2\,\mathrm{CCX}_{21\to0})^4$. Between $\sigma_0\sigma_2$ and itself the Toffoli gate $C = \mathrm{CCX}_{21\to0}$ is $T = \mathrm{CCX}_{30\to1}$, so K has the function of $TCTC$, the commutator of two Toffoli gates (Figure 12(a)). When the wires 2 and 3 are both 1 it cycles the states 01, 11, 10 of the wires 1 and 0, and it fixes every other state: it is the cycle of the positions 9, 10, 11. Swaps carry it to the other faces. Let $K_{10} = K$, $K_{01} = \sigma_0 K \sigma_0$, $K_{20} = \sigma_1 K \sigma_1$, $K_{02} = \sigma_1 K_{01} \sigma_1$, $K_{30} = \sigma_2 K_{20} \sigma_2$, and $K_{03} = \sigma_2 K_{02} \sigma_2$. Then K_{ab} cycles x, $x + e_a$, $x + e_a + e_b$, where x is the state in which wire a is 0 and the other wires are 1, and e_a flips wire a. If wire a changes from position i to $i+1$ and wire b from $i+1$ to $i+2$, the cycle

Ω_i is K_{ab} between X gates on the wires on which position i differs from x. For example, $\Omega_9 = K$, $\Omega_4 = \mathrm{X}_3 K_{01} \mathrm{X}_3$, and $\Omega_0 = \mathrm{X}_1\mathrm{X}_2\mathrm{X}_3\, K_{01}\, \mathrm{X}_3\mathrm{X}_2\mathrm{X}_1$. Written out, Ω_i has between 12 and 24 generators.

Lemma E.2 (Bridge.g-sem-gen). *For $0 \le i \le 13$, the circuit Ω_i carries position i to $i+1$, $i+1$ to $i+2$, and $i+2$ to i, and fixes the other positions.*

Proof. This is decided by evaluation at the sixteen positions, for each of the fourteen circuits. □

E.4 The First Step: the Translated Relation

This part follows the modules Translation, Lemmas, Bridge, Iso, and Presentation. The step is between the four-wire circuits and Lafont's words for $N = 16$.

The translations. The translation g reads a cycle as a circuit, $g(\omega_i) = \Omega_i$. The translation f reads a generator as a word in the cycles that carries out, on the positions, the permutation of the gate. On the gates on their own wires,

$$f(\mathrm{X}_0) = \omega_1\omega_0\,\omega_5\omega_4\,\omega_9\omega_8\,\omega_{13}\omega_{12}, \qquad f(\mathrm{CX}_{1\to0}) = \omega_3\omega_2\,\omega_{11}\omega_{10},$$
$$f(\mathrm{CCX}_{21\to0}) = \omega_5\omega_4\,\omega_7\omega_6\,\omega_9\omega_8, \qquad f(\sigma_0) = \omega_1\,\omega_3\omega_2\,\omega_4\,\omega_9\,\omega_{11}\omega_{10}\,\omega_{12},$$

and $f(\sigma_1)$ and $f(\sigma_2)$ are words of 12 and 24 cycles. Here $\omega_{i+1}\omega_i$ exchanges the positions i and $i+1$ and the positions $i+2$ and $i+3$, and for instance X_0 exchanges the positions $2m$ and $2m+1$ for every m. A generator further up goes to the word of the same gate one wire lower, between the words of the swaps that carry it there: $f(\mathrm{X}_{t+1}) = f(\sigma_t)\, f(\mathrm{X}_t)\, f(\sigma_t)$, and similarly for $\mathrm{CX}_{2\to1}$, $\mathrm{CX}_{3\to2}$, and $\mathrm{CCX}_{32\to1}$.

The translated relation. It is a relation on the four-wire circuits, with the structural rules of Figure 10 and two kinds of equations. The first are Lafont's relations read through g, that is (order), (near), (far), and (apart) with Ω in place of ω, in their 105 instances. The second are six *gate equations*, which say that each gate on its own wires is the circuit of its word: $\mathrm{X}_0 = g(f(\mathrm{X}_0))$, and likewise for $\mathrm{CX}_{1\to0}$, $\mathrm{CCX}_{21\to0}$, σ_0, σ_1, and σ_2. We write $u \approx_L w$ for equality modulo the translated relation.

Lemma E.3 (Bridge.f-sem, Bridge.g-sem). *For a four-wire circuit u, the word $f(u)$ permutes the positions as u maps the states. For a word v in the cycles, the circuit $g(v)$ maps the states as v permutes the positions.*

Proof. For a generator and for a cycle this is decided by evaluation at the sixteen positions; for the cycles it is Lemma E.2. The claims follow by induction on the word. Every statement about permutations is made position by position. □

Proposition E.4 (Iso.Theorem, Presentation). *The four facts hold for f and g between the translated relation and Lafont's relations. Hence the four-wire circuits modulo the translated relation form a group that $u \mapsto f(u)$ maps isomorphically onto Lafont's, and so onto* Alt(16) (presentation). *In particular, two four-wire circuits with the same function are equal modulo the translated relation* (complete), *and conversely* (sound).

Proof. The two facts toward the circuits are syntactic. An instance of one of Lafont's relations, read through g, is one of the 105 equations, by recursion on the instance with the number of shifts as a parameter (lemma-g-rel). A gate on its own wires is the circuit of its word by its gate equation. A generator further up is that gate between swaps, by the naturality of the swap, for example $\mathrm{X}_1 \approx_L \sigma_0\mathrm{X}_0\sigma_0$ and $\mathrm{CCX}_{32\to1} \approx_L \sigma_0\sigma_1\sigma_2\,\mathrm{CCX}_{21\to0}\,\sigma_2\sigma_1\sigma_0$, and f was defined to follow (lemma-g-f).

The two facts toward Lafont's words are semantic. Each equation of the translated relation holds for functions on states, which is decided by evaluation on the sixteen states; the equations with indices are decided for all their indices by one Boolean each. So by Lemma E.3 the words of its

two sides have the same permutation, and they are equal modulo Lafont's relations because his presentation is complete. In the same way, ω_i and $f(g(\omega_i)) = f(\Omega_i)$ have the same permutation. Hence no derivation between words in the cycles is written out.

Lafont's generators have inverse words, and the inverse of a generator x of the circuits is the circuit of the inverse of $f(x)$. The step gives the isomorphism, and Lafont's theorem identifies his group with Alt(16). Completeness follows because the isomorphism is injective and, by Lemma E.3, two circuits with the same function have words with the same permutation. □

E.5 At Most Three Wires

This part follows the modules BySymmetric, Width1, Width2, Width3, and ThreeWire. The library of EquaCert proves the Coxeter presentation of $\mathrm{Sym}(N)$ [28]. Its generators are the transpositions τ_d of d and $d+1$, for $0 \le d \le N-2$, and its relations are $\tau_d\tau_d = 1$, $\tau_d\tau_{d+1}\tau_d = \tau_{d+1}\tau_d\tau_{d+1}$, and $\tau_e\tau_d = \tau_d\tau_e$ for $e \ge d+2$. Two words in the τ_d with the same permutation are equal modulo these relations, and every permutation is that of a word.

On $k \le 3$ wires, number the $N = 2^k$ states along the reflected Gray code, as in Appendix E.3. The states numbered d and $d+1$ differ on one wire, and the *edge gate* Θ_d is the X gate on that wire, controlled by every other wire, each control filled or open according to the bit that the two states share on its wire. It exchanges the two states and fixes the others. On one wire, $\Theta_0 = \mathrm{X}_0$. On two wires, $\Theta_0 = \mathrm{CX}_{\bar{1}\to 0}$, $\Theta_1 = \mathrm{CX}_{0\to 1}$, and $\Theta_2 = \mathrm{CX}_{1\to 0}$. On three wires the seven edge gates are Toffoli gates,

$$\Theta_0 = \mathrm{CCX}_{\bar{2}\bar{1}\to 0}, \quad \Theta_1 = \mathrm{CCX}_{\bar{2}0\to 1}, \quad \Theta_2 = \mathrm{CCX}_{\bar{2}1\to 0}, \quad \Theta_3 = \mathrm{CCX}_{1\bar{0}\to 2},$$
$$\Theta_4 = \mathrm{CCX}_{21\to 0}, \quad \Theta_5 = \mathrm{CCX}_{20\to 1}, \quad \Theta_6 = \mathrm{CCX}_{2\bar{1}\to 0},$$

each written as $\mathrm{CCX}_{21\to 0}$ between X gates on its open controls and the swaps that bring its wires into place.

Proposition E.5 (complete$_1$–complete$_3$, presentation$_1$–presentation$_3$). *For $1 \le k \le 3$, the theory $\mathcal{T}$ is complete on k wires, G_k is a group, and $u \mapsto [\![u]\!]$ is an isomorphism from G_k onto* $\mathrm{Sym}(2^k)$.

Proof. The module BySymmetric carries out the step between the circuits on k wires modulo $\mathcal{T}$ and the Coxeter presentation of $\mathrm{Sym}(2^k)$, for any k, from the two facts toward the circuits and from the semantics of the two translations on generators. The translation g reads τ_d as Θ_d, and f reads a generator as a word in the transpositions. On three wires, for example,

$$f(\mathrm{X}_0) = \tau_0\tau_2\tau_4\tau_6, \qquad f(\mathrm{CX}_{1\to 0}) = \tau_2\tau_4, \qquad f(\mathrm{CCX}_{21\to 0}) = \tau_4, \qquad f(\sigma_0) = \tau_1\tau_2\tau_1\,\tau_4\tau_5\tau_4,$$

$f(\sigma_1)$ has 14 letters, and the generators further up are between the words of swaps, as in Appendix E.4.

The two facts toward the Coxeter words are semantic, as in Proposition E.4: $\mathcal{T}$ is sound by Proposition E.1, a word in the transpositions permutes the numbers as its circuit maps the states, which is decided by evaluation on the generators, and the Coxeter presentation is complete. The two facts toward the circuits are derivations from $\mathcal{T}$. They ask for the Coxeter relations between the Θ_d, and for each gate on its own wires to be the circuit of its word; a generator further up then follows by the naturality of the swap. On one wire the only relation is (T1). On two wires these are nine equations, and on three wires 33: seven of the form $\Theta_d\Theta_d \approx \varepsilon$, six braid relations, fifteen commutations, and five gates.

A Knuth–Bendix completion [44] finds the derivations. It starts from the instances of the rules on the given wires, oriented as rewrite rules by length and then alphabetically: seven instances on two wires, and 23 on three, of (T1)–(T6), of the laws of the swap, and of the commutations of generators on disjoint wires. Each rule that the completion adds comes from an overlap of two

earlier rules, and is thereby an equation derived from them. The completion stops as soon as the two sides of every target rewrite to the same word. The derivations of the targets use 16 added rules on two wires and 316 on three. Each added rule and each target becomes one derivation of the engine, 1,596 steps in all. On three wires only (T1)–(T6) are used, since (T7) needs four.

The step makes f an isomorphism from G_k onto the group of the Coxeter presentation, hence onto $\mathrm{Sym}(2^k)$, and completeness follows as in Proposition E.4. □

E.6 The Steps of a Four-Wire Derivation

This part follows the modules FourWire, Frames, and Letters. Each of the twelve generators on four wires lies on the bottom three wires, on the top three, or on both.

Lemma E.6 (three-wire steps, bot, top). *Two circuits on the wires 0, 1, 2 of four wires that denote the same function are equal modulo $\mathcal{T}$, and so are two circuits on the wires 1, 2, 3.*

Proof. As circuits on three wires they are equal by Proposition E.5. The structural rules carry the derivation to four wires, with an idle wire on top or, by (shift), below. That the two functions agree is decided by evaluation on the eight states. □

Thirteen instances of the rules involve all four wires and are not three-wire steps. They are (T7); the naturality of the swap at the Toffoli gate, $\mathrm{CCX}_{21\to0}\,\sigma_2\sigma_1\sigma_0 \approx \sigma_2\sigma_1\sigma_0\,\mathrm{CCX}_{32\to1}$; and eleven commutations of a generator that touches wire 0 with a generator that touches wire 3 and none of its wires: X_0 with X_3, $\mathrm{CX}_{3\to2}$, σ_2, and $\mathrm{CCX}_{32\to1}$; each of $\mathrm{CX}_{1\to0}$ and σ_0 with X_3, $\mathrm{CX}_{3\to2}$, and σ_2; and $\mathrm{CCX}_{21\to0}$ with X_3. Every four-wire derivation of the formalization is built from three-wire steps and these thirteen equations.

Symmetry words. A *symmetry word* is a word in $\mathrm{X}_0, \ldots, \mathrm{X}_3$ and $\sigma_0, \sigma_1, \sigma_2$. These circuits carry out the 384 symmetries of the 4-cube, which permute the wires and complement some of them. The reversed word p^{-1} of a symmetry word p has the inverse function. A *normal form* is a triple (q, e, r) of a symmetry word q on the bottom three wires, a bit e, and one of the four words $r \in \{\varepsilon,\ \sigma_2,\ \sigma_2\sigma_1,\ \sigma_2\sigma_1\sigma_0\}$, which bring each of the four wires to the top. Its circuit is $q\,\mathrm{X}_3^e\,r$.

Lemma E.7 (hstep). *Every symmetry word p has a normal form $\mathrm{nf}(p) = (q, e, r)$ with $p \approx q\,\mathrm{X}_3^e\,r$. Hence two symmetry words are equal modulo $\mathcal{T}$ if their normal forms have the same e, the same r, and words q with the same function.*

Proof. The normal form is computed one letter at a time, from $(\varepsilon, 0, \varepsilon)$. A table gives, for each of the four words r and each of the seven letters y, a letter or the empty word b on the bottom three wires, a bit e', and a word r' with $r\,y \approx b\,\mathrm{X}_3^{e'}\,r'$. For example, $\sigma_2\sigma_1\,\mathrm{X}_1 \approx \mathrm{X}_3\,\sigma_2\sigma_1$ and $\sigma_2\sigma_1\sigma_0\,\sigma_1 \approx \sigma_0\,\sigma_2\sigma_1\sigma_0$. Each of the 28 entries is derived in at most three steps, which are three-wire steps and commutations. Since X_3 commutes with every generator on the bottom three wires, by the structural rules, appending y to $q\,\mathrm{X}_3^e\,r$ gives $(qb)\,\mathrm{X}_3^{e+e'}\,r'$, which proves the first claim by induction. For the second, the two words q are equal by a three-wire step. In the formalization the comparison of two normal forms is a Boolean, which Agda evaluates for each pair of words to which the lemma is applied. □

Frames. For a symmetry word p and a circuit b on the bottom three wires, the circuit $p\,b\,p^{-1}$ is b *in the frame* p. It acts as b on the wires that p brings to the bottom, with the polarities that p sets.

Lemma E.8 (change of frame, frame). *Let p and p' be symmetry words and b and b' circuits on the bottom three wires. Suppose that $\mathrm{nf}(p^{-1}p') = (q, e, \varepsilon)$ and $\mathrm{nf}(p'^{-1}p) = (\bar{q}, e, \varepsilon)$, and that $q\,b'\,\bar{q}$ has the function of b. Then $p\,b\,p^{-1} \approx p'\,b'\,p'^{-1}$.*

Proof. By Lemma E.7, $p\,p^{-1} \approx \varepsilon$, so $p' \approx p\,(p^{-1}p') \approx p\,q\,\mathrm{X}_3^e$ and $p'^{-1} \approx (p'^{-1}p)\,p^{-1} \approx \bar{q}\,\mathrm{X}_3^e\,p^{-1}$. The gate X_3 commutes with b' and with $\bar{q}$, and $\mathrm{X}_3\mathrm{X}_3 \approx \varepsilon$, so

$$p'\,b'\,p'^{-1} \approx p\,q\,\mathrm{X}_3^e\,b'\,\bar{q}\,\mathrm{X}_3^e\,p^{-1} \approx p\,(q\,b'\,\bar{q})\,p^{-1} \approx p\,b\,p^{-1},$$

the last by a three-wire step. The hypotheses are one Boolean, evaluated for each use. □

The hypothesis on the normal forms says that p and p' bring the same wire to the top. The two lemmas are the only general facts about four wires that the formalization proves without a search. In the generated derivations a step is a three-wire step, one of the thirteen equations, an instance of Lemma E.7 or of Lemma E.8, or an equation derived before.

E.7 Placed Gates and Pair Lemmas

This part follows the generated modules Four.Part1–Four.Part14 and the scripts that write them. The search works with the 76 *placed gates* on four wires: four X_t, the 24 gates $\mathrm{CX}_{c\to t}$ with a filled or an open control, and the 48 gates $\mathrm{CCX}_{ab\to t}$ with each control filled or open. As a circuit, X_t is a generator, and every other placed gate is the base gate $\mathrm{CX}_{1\to 0}$ or $\mathrm{CCX}_{21\to 0}$ in a frame: X gates on its open controls, followed by the swaps that bring its target and its controls to the wires 0, 1, and 2. By Lemma E.8, a placed gate conjugated by a symmetry word is the placed gate that the symmetry carries it to, whatever the frames.

A *macro derivation* rewrites a word of placed gates, in which gates on disjoint wires may be interchanged, by three kinds of move. A *narrow* move replaces consecutive gates on three of the four wires by gates on the same wires with the same function. The other two use a *pair lemma*: an equation $(g\,h)^2 = k$ for two placed gates g and h that together touch all four wires, where k is the identity or a placed gate. A *pair* move rewrites $g\,h$ to $k\,h\,g$ or to $h\,g\,k$, and an *expand* move rewrites k to $g\,h\,g\,h$. Each move becomes a few steps of the engine.

- *Narrow.* The frame of the wires 0, 1, 2 is empty, and those of the triples 0, 1, 3, then 0, 2, 3, then 1, 2, 3 are σ_2, $\sigma_1\sigma_2$, and $\sigma_0\sigma_1\sigma_2$. Each gate of the block is brought to the frame of the triple by Lemma E.8, the frames between consecutive gates cancel by Lemma E.7, a three-wire step on the bottom wires replaces the block, and the new gates are brought back. A CNOT has two idle wires, and when its frame brings the wrong one to the top, the frame is first extended by σ_2, which commutes with $\mathrm{CX}_{1\to 0}$.
- *Interchange.* An X gate passes a placed gate that does not touch its wire: it moves through the frame by Lemma E.7, where it becomes X_2 or X_3, passes the base gate by a three-wire step or a commutation, and moves back. Two CNOTs on complementary pairs of wires are $\mathrm{CX}_{1\to 0}$ and $\mathrm{CX}_{3\to 2}$ in a common frame, where they commute by a structural rule.
- *Pair and expand.* From $(g\,h)^2 \approx k$ and the involutions $g\,g \approx h\,h \approx k\,k \approx \varepsilon$, which are three-wire steps inside frames, one gets $g\,h \approx k\,h\,g$ and $g\,h \approx h\,g\,k$ by inserting $g\,g$ and $h\,h$.

There are 2,496 ordered pairs (g, h) with a pair lemma, and the symmetries of the cube permute them in 12 orbits. Table 5 lists a representative of each.

Proposition E.9 (pair lemmas). *The representatives of the twelve orbits hold modulo $\mathcal{T}$, and so does every pair lemma that a derivation of the formalization uses.*

Proof. *Equation* (T7). Let $T = \mathrm{CCX}_{32\to 1}$, $c = \mathrm{CX}_{1\to 0}$, and $Q = \mathrm{CCX}_{32\to 0}$. The naturality of the swap at the Toffoli gate and Lemma E.7 identify the generator T with $\mathrm{CCX}_{21\to 0}$ in the frame $\sigma_0\sigma_1\sigma_2$, and $\sigma_0 T \sigma_0$ with the placed gate Q. Then (T7), which reads $TcT \approx Qc$, gives $(T\,c)^2 \approx Q$ after multiplying by c, an involution.

The other order. If $(g\,h)^2 \approx k$, then $(h\,g)^2 \approx k$: since $k\,k \approx \varepsilon$, we have $hghg \approx hghg\,(ghgh)\,k \approx k$, where the middle cancels pair by pair.

Table 5. The twelve orbits of pair lemmas in the order in which they are derived, down the left column and then the right, with the number of moves of the macro derivation of the representative. The first is (T7).

$g,\ h$	$(g\,h)^2$	moves	$g,\ h$	$(g\,h)^2$	moves
$\mathrm{CX}_{1\to0},\ \mathrm{CCX}_{32\to1}$	$\mathrm{CCX}_{32\to0}$	–	$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{32\to1}$	$\mathrm{CCX}_{32\to0}$	21
$\mathrm{CX}_{1\to0},\ \mathrm{CCX}_{32\to0}$	identity	7	$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{3\bar{2}\to1}$	identity	12
$\mathrm{CX}_{1\to0},\ \mathrm{CCX}_{31\to2}$	identity	6	$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{21\to3}$	identity	6
$\mathrm{CX}_{1\to0},\ \mathrm{CCX}_{3\bar{1}\to2}$	identity	7	$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{\bar{2}1\to3}$	identity	6
$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{31\to0}$	identity	54	$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{\bar{2}\bar{1}\to3}$	identity	9
$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{3\bar{1}\to0}$	identity	8	$\mathrm{CX}_{1\to0},\ \mathrm{CCX}_{30\to2}$	$\mathrm{CCX}_{31\to2}$	7

Transport. If $(g\,h)^2 \approx k$ and w is a symmetry word, then each of $w\,g\,w^{-1}$, $w\,h\,w^{-1}$, and $w\,k\,w^{-1}$ is the placed gate that the symmetry gives, by Lemma E.8, and the frames between them cancel. So the lemma holds for the pair that w carries (g, h) to. The formalization derives in this way each instance that a derivation uses, from the representative of its orbit.

The representatives. For each orbit after the first, a search finds a macro derivation that reduces the word $g\,h\,g\,h\,k$ of the representative to the empty word, with pair and expand moves only for the orbits derived before. Table 5 gives the order and the sizes. The second orbit, for example, follows in a few steps, which the formalization also derives without a search (comm-cx0-Q): since $Q \approx TcTc$ and Q is an involution, $c\,Q\,c \approx c\,(TcTc)\,c \approx cTcT \approx Q^{-1} \approx Q$, so c commutes with Q. The last orbit of the table is not found while only the first two are known, and is found at once after the others. □

E.8 The Relations between the Cycles

This part follows the generated modules Four.Part1–Four.Part14 and Four.Relations.

The forms of a cycle. With $T = \mathrm{CCX}_{30\to1}$ and $C = \mathrm{CCX}_{21\to0}$ as in Appendix E.3, the circuit K is equal modulo $\mathcal{T}$ to $T\,C\,T\,C$ as a word of placed gates, by two uses of the commutation of σ_0 and σ_2. Each Ω_i is K between a symmetry word and its reverse, so by Lemma E.8 it is equal to a word $T_iC_iT_iC_i$ of four placed Toffoli gates. Let wire a change from position i to $i+1$, wire b from $i+1$ to $i+2$, and let c and d be the other two wires. Then one of the two gates flips wire b where wire a is as in position $i+1$ and wire d as in position i, and the other flips wire a where wire b is as in position $i+1$ and wire c as in position i. The wires c and d may be given to the two gates either way round, so a cycle has two such words, its *forms*. For Ω_9 they are $(\mathrm{CCX}_{20\to1}\,\mathrm{CCX}_{31\to0})^2$ and $(\mathrm{CCX}_{30\to1}\,\mathrm{CCX}_{21\to0})^2$.

Lemma E.10 (the two forms). *The two forms of each cycle are equal modulo $\mathcal{T}$.*

Proof. For Ω_9, a search finds a macro derivation of 17 moves. A symmetry of the cube carries the positions 9, 10, 11 to i, $i+1$, $i+2$, and it carries the two forms of Ω_9 to the two forms of Ω_i, so the equation is transported as in Proposition E.9. □

Shapes. A symmetry of the cube carries three corners of a face to three corners of a face. The 105 instances of Lafont's relations fall into 24 orbits under the symmetries, their *shapes*: one for (order), two for (near), two for (far), and 19 for (apart).

Proposition E.11 ($\mathrm{order}^\mathsf{T}$, near^T, far^T, $\mathrm{apart}^\mathsf{T}$). *The* 105 *instances of* (order), (near), (far), *and* (apart) *between the circuits Ω_i hold modulo $\mathcal{T}$.*

Proof. *One relator for each shape.* For an instance $l = r$ of a shape, with each cycle in one of its forms, the *relator* is the word $l\,r^{-1}$ of placed gates, where r^{-1} is r reversed. It has 12 letters for

(order), 20 for (near), 24 for (far), and 16 for (apart). For every shape but (order), a best-first search on the length of the word finds a macro derivation that reduces the relator to the empty word, in 22 to 123 moves. The forms matter to the search: for four shapes of (apart) the relator has a cycle in its second form, with which the search ends in seconds, while with both cycles in their first form it exceeds its memory bound.

Order. The search does not find $K^3 \approx \varepsilon$ directly. Let $T = \mathrm{CCX}_{20\to1}$ and $C = \mathrm{CCX}_{31\to0}$, so that $K \approx (T\,C)^2$, let $t = \mathrm{CX}_{0\to1}$, and let $z = C\,t\,C\,t\,C$, a word on the wires 0, 1, and 3. The search finds $T\,C\,T\,C\,T \approx t\,C\,T\,C\,t$ in 27 moves and $T\,z \approx z\,T$ in five. Then

$$K^3 \approx (TCTCT)\,C\,(TCTCT)\,C \approx t\,C\,T\,z\,T\,C\,t\,C \approx t\,C\,z\,T\,T\,C\,t\,C \approx t\,C\,z\,C\,t\,C,$$

and the last word is on three wires and has the function of the identity, so it is reduced by narrow moves. The composed derivation has 77 moves.

From a shape to its instances. Let a symmetry carry the cycles of the shape's instance to those of another instance. Conjugating the relator by a symmetry word gives, gate by gate and by Lemma E.8, a relator of the other instance in some forms of its cycles. Lemma E.10 changes these to the forms that the words Ω_i unfold to. Finally $l \approx l\,r^{-1}r \approx r$, where $r^{-1}r$ is inserted pair by pair. All of this is generated: the 24 relators, the lemma of the two forms for the fourteen cycles, and the 105 instances are equations of the engine between lists of generators. □

In Agda each family is a function of its indices with a Boolean guard, so that the indices out of range need no case.

E.9 The Gates as Products of Cycles

This part follows Four.Relations and the module Equivalence. For even $i \le 12$, the positions $i, \ldots, i+3$ are the four corners of a face of the cube. The two cycles on them generate the twelve even permutations of these corners, and two of these are Toffoli gates: $\Omega_{i+1}\Omega_i$ exchanges i with $i+1$ and $i+2$ with $i+3$, and $\Omega_{i+1}\Omega_i\Omega_i\Omega_{i+1}$ exchanges i with $i+3$ and $i+1$ with $i+2$. Write t_i and t'_i for the placed Toffoli gates with these functions:

$$\begin{array}{llll} t_0 = \mathrm{CCX}_{\bar3\bar2\to0}, & t_4 = \mathrm{CCX}_{\bar32\to0}, & t_8 = \mathrm{CCX}_{32\to0}, & t_{12} = \mathrm{CCX}_{3\bar2\to0}, \\ t_2 = \mathrm{CCX}_{\bar31\to0}, & t_6 = \mathrm{CCX}_{2\bar1\to0}, & t_{10} = \mathrm{CCX}_{31\to0}, & \\ t'_0 = \mathrm{CCX}_{\bar3\bar2\to1}, & t'_4 = \mathrm{CCX}_{\bar32\to1}, & t'_8 = \mathrm{CCX}_{32\to1}, & t'_{12} = \mathrm{CCX}_{3\bar2\to1}, \\ t'_2 = \mathrm{CCX}_{\bar31\to2}, & t'_6 = \mathrm{CCX}_{2\bar1\to3}, & t'_{10} = \mathrm{CCX}_{31\to2}. & \end{array}$$

Lemma E.12. *For even $i \le 12$, $\Omega_{i+1}\Omega_i \approx t_i\,t'_i$, hence $t'_i \approx \Omega_{i+1}\Omega_i\Omega_i\Omega_{i+1}$. Moreover $\Omega_1\,t_2\,\Omega_4 \approx \mathrm{CCX}_{\bar30\to1}\,t_2\,\mathrm{CCX}_{\bar30\to1}$ and $\Omega_9\,t_{10}\,\Omega_{12} \approx \mathrm{CCX}_{30\to1}\,t_{10}\,\mathrm{CCX}_{30\to1}$, and $\mathrm{CCX}_{21\to0} \approx t_4\,t_6\,t_8$.*

Proof. For each of the seventeen equations, a search finds a macro derivation of its relator, of 4 to 12 letters, in at most 27 moves. □

Lemma E.13 (gate-X$^\mathsf{T}$, gate-CX$^\mathsf{T}$, gate-CCX$^\mathsf{T}$, gate-Ex$_0{}^\mathsf{T}$, gate-Ex$_1{}^\mathsf{T}$, gate-Ex$_2{}^\mathsf{T}$). *Each of X_0, $\mathrm{CX}_{1\to0}$, $\mathrm{CCX}_{21\to0}$, σ_0, σ_1, and σ_2 is equal modulo $\mathcal{T}$ to the circuit $g(v)$ of some word v in the cycles.*

Proof. By Lemma E.12 each t_i and t'_i is such a circuit, and so is $\mathrm{CCX}_{21\to0}$. The other gates are products of these by one three-wire step each, in the frame of three of the four wires. On the wires 0, 2, 3, the gate X_0 is $t_0\,t_4\,t_8\,t_{12}$, the Toffoli gates onto wire 0 for the four values of the wires 3 and 2. On the wires 0, 1, 3, the gate $\mathrm{CX}_{1\to0}$ is $t_2\,t_{10}$, and σ_0 is the product of the two controlled swaps of Lemma E.12, which exchange the wires 0 and 1 where wire 3 is 0 and where it is 1. On the wires 1, 2, 3, we have $\mathrm{CX}_{2\to1} = t'_4\,t'_8$ and $\mathrm{CX}_{1\to2} = t'_2\,t'_{10}$, so $\sigma_1 = \mathrm{CX}_{2\to1}\,\mathrm{CX}_{1\to2}\,\mathrm{CX}_{2\to1}$ is a product of six of the t'_i. Also $\mathrm{X}_1 = t'_0\,t'_4\,t'_8\,t'_{12}$, which gives the open controls in $\mathrm{CX}_{3\to2} = t'_{10}\,(\mathrm{X}_1\,t'_{10}\,\mathrm{X}_1)$ and $\mathrm{CX}_{2\to3} = (\mathrm{X}_1\,t'_6\,\mathrm{X}_1)\,t'_6$, so $\sigma_2 = \mathrm{CX}_{3\to2}\,\mathrm{CX}_{2\to3}\,\mathrm{CX}_{3\to2}$ is a product of thirty of the t'_i. For X_0, $\mathrm{CX}_{1\to0}$,

$\mathrm{CCX}_{21\to 0}$, and σ_0 the word v reached is f of the gate; for σ_1 and σ_2 it has 24 and 120 cycles, where f has 12 and 24. □

Lemma E.14 (to-f). *If a four-wire circuit c is equal modulo $\mathcal{T}$ to $g(v)$ for some word v in the cycles, then $c \approx g(f(c))$.*

Proof. By Proposition E.1 and Lemma E.3, v permutes the positions as c maps the states, and so does $f(c)$. So v and $f(c)$ are equal modulo Lafont's relations, by his completeness. By Proposition E.11 each instance of his relations, read through g, holds modulo $\mathcal{T}$, so g carries that derivation to $g(v) \approx g(f(c))$ (Lemmas.FromCycles). □

Proposition E.15 (lemma-g-f^{T}). *The six gate equations of the translated relation hold modulo $\mathcal{T}$, and every generator x on four wires satisfies $x \approx g(f(x))$.*

Proof. For the six gates on their own wires, combine Lemmas E.13 and E.14. A generator further up is the gate between swaps by the naturality of the swap, hence the circuit of some word in the cycles, and Lemma E.14 applies again. □

E.10 The Second Step and Completeness

This part follows the modules Equivalence and MainTheorem.

Proposition E.16 (s⇒t, t⇒s, rules≅relations). *Two four-wire circuits are equal modulo the translated relation if and only if they are equal modulo $\mathcal{T}$.*

Proof. The step is between two relations on the same circuits, with the identity translations, so its third and fourth facts are trivial and the other two say that each relation derives the equations of the other. The structural rules are the same on both sides. The equations of the translated relation hold modulo $\mathcal{T}$ by Propositions E.11 and E.15. Conversely, by Proposition E.1 the two sides of an equation that holds modulo $\mathcal{T}$ denote the same function, so they are equal modulo the translated relation, which is complete by Proposition E.4. □

Proof of Theorem 6.1. Soundness is Proposition E.1. For $k \le 3$, completeness and the group G_k are Proposition E.5. On four wires, two circuits with the same function are equal modulo the translated relation by Proposition E.4, hence modulo $\mathcal{T}$ by Proposition E.16 (complete^{R}). By the same two propositions, G_4 is the group of the circuits modulo the translated relation, which reading a circuit as the permutation that it carries out on the positions maps isomorphically onto $\mathrm{Alt}(16)$ (presentation^{R}). The formalization also proves the four facts of the first step for $\mathcal{T}$ itself, with the same f and g (Theorem-Rules-iso-Cycles): toward Lafont's words by soundness and his completeness, and toward the circuits by Propositions E.11 and E.15. □

E.11 The Canonical Form

This part follows the module NormalForm and the library's Alternating. The stairs $s_d(j)$ and Lafont's canonical forms among N points are those of Section 6: a canonical form is the product of one stairs for each $d \le N-3$, the highest first, with $d \le j_d \le N-1$, so there are $N(N-1)\cdots 3 = N!/2$ canonical forms. We number the points from 0, where Lafont numbers them from 1, and we write products of permutations in the order of circuits, where he writes them in the order of composition. The stairs to the last point repeats its last cycle; Lafont treats this case as a separate type of canonical form. The targets of an even permutation π are computed one stairs at a time: $\pi_0 = \pi$, and for each d in turn, $j_d = \pi_d(d)$ and $\pi_{d+1} = \pi_d\, s_d(j_d)^{-1}$, which fixes the points $0, \ldots, d$. At the end, π_{N-2} is an even permutation that fixes all points but the last two, so it is the identity, and π is the permutation of the canonical form with the targets $j_0, \ldots, j_{N-3}$. The library proves Lafont's Lemmas 6 and 8.

Lemma E.17 (Lafont; Surjectivity.canonical, UniqueNormalForm, Normalization). *Every even permutation of N points is the permutation of exactly one canonical form, and every word in the cycles is equal, modulo Lafont's relations, to a canonical form.*

Proof. For existence, the stairs from 0 to $\pi(0)$ is divided out of an even permutation π. What is left fixes the point 0 and is an even permutation of the other points, to which the construction applies again. This is the computation of the targets given above. For uniqueness, the first target of a canonical form is where its permutation carries 0, and cancelling the last stairs leaves two canonical forms on one point fewer. For the last claim, Lafont gives seven rewriting rules that follow from his relations and a lemma that carries a cycle through a stairs (his Lemma 7), and he leaves the induction to the reader: one cycle at a time is pushed through a canonical form, as in his proof for the symmetric groups. The library follows his proofs of existence and uniqueness and carries out this induction. □

Write $S_d(j)$ for the circuit $g(s_d(j))$ of a stairs. The canonical circuits of Section 6 are g of Lafont's canonical forms for $N = 16$, letter for letter (canon-g): $\mathrm{Can}(j_0, \ldots, j_{13}) = S_{13}(j_{13}) \cdots S_1(j_1)\, S_0(j_0)$. In Agda a tuple of targets is a tuple of stairs, nf w is the canonical form that Lemma E.17 gives for the word $f(w)$, and canon is g of a canonical form.

Proof of Theorem 6.2. *Existence* (normal-form^{R}, nf-sem). Let u be a four-wire circuit and v the canonical form equal to $f(u)$ modulo Lafont's relations. His relations are sound, so v has the permutation of $f(u)$, which by Lemma E.3 is the permutation that u carries out, and $g(v)$ carries out the same permutation. So u and the canonical circuit $g(v)$ have the same function, and they are equal modulo $\mathcal{T}$ by Theorem 6.1. By the uniqueness in Lemma E.17, v is the canonical form that the existence proof computes from that permutation, so its targets are those of the permutation.

Uniqueness and the bijection (canon-unique, canon-onto, nf-canon, nf-cong^{R}). Two canonical circuits with the same function are g of two canonical forms with the same permutation, by Lemma E.3, and these are the same by Lemma E.17. Every even permutation of the positions is the permutation of a canonical form, whose circuit carries it out. If u is equal modulo $\mathcal{T}$ to a canonical circuit, then by Proposition E.1 they carry out the same permutation, so that circuit is the one above. The count $16 \cdot 15 \cdots 3 = 16!/2$ is the number of tuples of targets. □

Example. The canonical circuit that a circuit is equal to is thus read off its permutation. The Toffoli gate $\mathrm{CCX}_{21\to 0}$ exchanges the states 0110 and 0111, which are the positions 4 and 5, and the states 1111 and 1110, the positions 10 and 11, and it fixes the other twelve positions. Let π be this permutation. It fixes the positions below 4, so $j_d = d$ and $\pi_{d+1} = \pi$ for $d \le 3$. Then $j_4 = \pi(4) = 5$, and dividing by the stairs $s_4(5) = \omega_4$ leaves π_5, which exchanges 5 with 6 and still 10 with 11. In the same way $j_d = d + 1$ for $d = 5, \ldots, 8$, the first exchange moving up one position at each step, until π_9 is the cycle ω_9 itself, so that $j_9 = 10$, π_{10} is the identity, and $j_d = d$ for $d \ge 10$. The canonical form of π is therefore $\omega_9\, \omega_8\, \omega_7\, \omega_6\, \omega_5\, \omega_4$ (Figure 11(c)), and the gate is equal modulo $\mathcal{T}$ to the canonical circuit

$$S_9(10)\, S_8(9)\, S_7(8)\, S_6(7)\, S_5(6)\, S_4(5) = \Omega_9\, \Omega_8\, \Omega_7\, \Omega_6\, \Omega_5\, \Omega_4,$$

six stairs of one cycle each, the other eight being empty (Figure 12(b)). With every cycle written out as it is defined in Appendix E.3, this circuit has 104 generators, twelve of them in Ω_9. Other circuits have longer stairs. The swap σ_1, for example, is equal to the canonical circuit

$$S_{10}(12)\, S_8(12)\, S_4(6)\, S_2(6) = (\Omega_{10}\, \Omega_{11})\, (\Omega_8\, \Omega_9\, \Omega_{10}\, \Omega_{11})\, (\Omega_4\, \Omega_5)\, (\Omega_2\, \Omega_3\, \Omega_4\, \Omega_5).$$

Lafont's rewriting also computes the canonical form, but on the word of a circuit it passes through millions of letters, and the formalization never runs it.

F Details of the Corrections

This appendix states the two corrections of Section 5 in a form that can be checked by hand, and lists the main misprints that the formalizations found in their sources.

F.1 The Gap in the Proof of Theorem 4.14 of Hadamard-Π

References are to the extended version of [31], whose Appendix A proves Theorem 4.14, and indices start at 1, as there. Applied to a vector x, the matrix $\mathrm{H}_{[i,j]}$ replaces the entries x_i and x_j by $(x_i + x_j)/\sqrt{2}$ and $(x_i - x_j)/\sqrt{2}$, and a product acts with its rightmost factor first. An element $a + b\sqrt{2}$ of $\mathbb{Z}[\sqrt{2}]$ is *odd* if a is odd.

Lemma A.16 concerns a unit vector $\hat{v}$ over $\mathbb{Z}[1/\sqrt{2}]$ and a $k \geq 2$ such that $\sqrt{2}^k \hat{v}$ has entries in $\mathbb{Z}[\sqrt{2}]$, exactly four of them odd, namely the first four, which are congruent to $(1 + \sqrt{2},\ 1,\ 1 + \sqrt{2},\ 1)$ or to $(1,\ 1 + \sqrt{2},\ 1,\ 1 + \sqrt{2})$ modulo 2. Let v consist of the first four entries of $\hat{v}$. The lemma states that if $\sqrt{2}^{k-1}\, \mathrm{H}_{[1,4]}\mathrm{H}_{[2,3]}\mathrm{H}_{[2,4]}\mathrm{H}_{[2,3]}\mathrm{H}_{[1,4]}\, v$ has entries in $\mathbb{Z}[\sqrt{2}]$, then so does $\sqrt{2}^{k-1}\, \mathrm{H}_{[1,3]}\mathrm{H}_{[2,4]}\mathrm{H}_{[1,2]}\mathrm{H}_{[1,3]}\mathrm{H}_{[2,4]}\, v$. It fails for $k = 4$ in dimension 6, on

$$4\,\hat{v} = (-1 - \sqrt{2},\ 1,\ -1 - \sqrt{2},\ 1,\ \sqrt{2},\ -2 + \sqrt{2}).$$

The squares of these entries add up to 16, so $\hat{v}$ is a unit vector, and the first four entries are odd and congruent to $(1 + \sqrt{2}, 1, 1 + \sqrt{2}, 1)$. The first product gives $(0,\ 0,\ -1 - \sqrt{2},\ -1)$, and the second gives $(-\frac{\sqrt{2}}{2},\ -1 - \frac{\sqrt{2}}{2},\ -\frac{\sqrt{2}}{2},\ -1 - \frac{\sqrt{2}}{2})$, which is not in $\mathbb{Z}[\sqrt{2}]^4$. Lemma A.17 is the same statement with $\mathrm{H}_{[3,4]}$ in place of $\mathrm{H}_{[2,4]}$ in the two words, and it fails in the same way on $4\,\hat{v} = (1 + \sqrt{2},\ -1,\ 1 + \sqrt{2},\ 1,\ -\sqrt{2},\ 2 - \sqrt{2})$.

Items (c) and (d) of Lemma A.19 rest on these two lemmas, and Main Case 9 of the Main Lemma rests on Lemma A.19. That case is not covered for the unit vector $\hat{s}$ in dimension 9 with

$$\sqrt{2}^7\, \hat{s} = (1 + 3\sqrt{2},\ 1,\ -3\sqrt{2},\ 1 + 3\sqrt{2},\ 2,\ 5,\ -2 + \sqrt{2},\ -4 + 2\sqrt{2},\ -2 - 2\sqrt{2}),$$

whose odd entries are the first, second, fourth, and sixth: none of the eight diagrams that the proof of Lemma A.19 offers stays below the level of $\hat{s}$, as the Main Lemma requires. The step that the proof leaves open is $\mathrm{H}_{[c,d]}$ on two odd entries that are not congruent modulo 2, when exactly four entries are odd. It is Case 3.4 of [21], which the formalization follows [5].

F.2 The Relation Missing from Theorem 5.2 of Toffoli–Hadamard

Let J be the 8×8 matrix whose entries are all 1, and let $R = I - J/4$, the reflection that negates the vector of all ones. Then $R^2 = I$, since $J^2 = 8J$, and R commutes with every permutation matrix. Read each gate g of Figure 4 of [3] as RgR, and the two further generators of its Figure 7, namely $\mathrm{K}_{[0,1,2,3]}$ and CCZ, as themselves, and take both figures with the misprints of Table 6 corrected. Every relation of Figure 4 still holds, being a matrix identity conjugated by R. So does every relation of Figure 7, since its other gates are permutation matrices, which are read as themselves. The relation r_{K} equates $I \otimes \mathrm{K}$ with a word in $\mathrm{X} \otimes I \otimes I$ and $\mathrm{K}_{[0,1,2,3]}$. Its left-hand side is read as $R\,(I \otimes \mathrm{K})\,R$, whose entry in the first row and column is $1/2 - 2/4 - 2/4 + 4/16 = -1/4$, and its right-hand side is read as itself, with the entry $1/2$ there. So r_{K} does not follow from the two figures. In the formalization, this is `r_K-independent` [6].

F.3 Misprints by Source

Table 6 lists the main things that the formalizations found besides the two corrections above. They were found by the checks of Section 3.2 or when a step of a proof failed, and the formalizations

state the corrected forms. The table leaves out smaller slips, such as a wrong label on a step of a derivation. For the other sources of Table 1 we recorded none.

Source	What the formalization found
Amy et al. [2024]	Seven equations are false as printed: (33) of Figure 4, (97) and (100) of Figure 7, and four of Sections 5.5 to 5.7, among them (r_{CX}) and a decomposition of CCZ. Each is refuted in Agda by one matrix entry. The proof of Lemma 6.1 treats six of the twenty-five generators; the others need an induction.
Makary et al. [2021]	Eleven of the 137 typed relations of Appendix B are false as printed: six have a different right-hand side, and five lack a Pauli gate.
Selinger [2015]	Three steps of the supplement's derivations cite the wrong equation. The relations of the paper all hold.
Li et al. [2021]	Relation (7d) of Table 2 has its two cases exchanged.
Li et al. [2025]	The third row of Figure 25 has one CZ in its residual circuit, not two.
Blake [2026c]	The scalar of the qutrit rule (E) in Figure 4 is ω_{12}, not ω_{12}^7. One interpretation of Figure 8, which separates a rule on two wires, does not preserve a rule on three; the claim stands, since a derivation on two wires uses no rule on three.
Clément [2026]	Equations (36) and (37) hold with the negations on the controls that are 1; we read the paper as placing them on those that are 0. Equations (30), (34), and (42) need the two indices of each of their letters to differ, which the figure leaves implicit.

Table 6. The main misprints and slips that the formalizations found in their sources, besides the two corrections of Section 5.